



TRAINOSYS

Training the Future Today



About The Trainer

- ❖ Senior Software Developer with 20 years of expertise in software development, server administration, database design, and cybersecurity.
- ❖ Brings 9 years of teaching experience both locally and internationally. Skilled in Artificial Intelligence and recognized for being collaborative, approachable, and effective in team environments.

Training Schedule

Start	Coffee Break (15mins)	Lunch Break	Continued	Coffee Break (15mins)	End
9:00 am	10:30 am – 10:45 am	12 noon – 1 pm	1pm	3:00 pm - 3:15 pm	5:00 pm

- In the event of a power outage, a standby generator is available and can operate for up to 7 hours.
- In case of internet disruptions, a backup connection is ready. If the backup cannot be sustained, a rescheduling may be necessary. We will inform you as soon as possible.

Advanced MySQL Programming

Training Course

Training Overview

- This course focusing on performance tuning, query optimization, and real-world database problem solving scenarios
- Participants will learn complex joins, subqueries, indexing strategies, and transaction control to design efficient, scalable, and maintainable database solutions
- Encryption, metadata lock avoidance, logging analysis, and configuration-based performance tuning in environments
- Troubleshoot slow queries, optimize workloads, and implement best practices for secure and high-performing MySQL systems

Training Objectives

- Apply advanced techniques to solve real-world database problems
- Design efficient joins and avoid duplicate or incorrect results
- Optimize queries using indexing and execution plan analysis
- Implement subqueries and CTEs for complex data retrieval scenarios
- Manage transactions ensuring data consistency and concurrency
- Analyze logs to troubleshoot performance issues and system behavior
- Apply encryption and configuration tuning for secure database

Training Audience

- Database Developers
- Backend Developers / Software Engineers
- Database Administrators (DBAs)
- Data Analysts / BI Developers
- System Architects / Technical Leads
- DevOps Engineers / Platform Engineers
- IT Professionals transitioning to Database Roles

Training Pre-requisites

- Knowledge in SQL queries (SELECT, INSERT, UPDATE, DELETE)
- Familiarity with JOINS, GROUP BY, and simple filtering conditions
- Knowledge of relational database concepts (tables, keys, relationships)
- Experience using any SQL client (e.g., MySQL Workbench or CLI)
- Basic understanding of transactions (COMMIT and ROLLBACK)
- Familiarity with command line or terminal usage
- Basic knowledge of Docker concepts (containers, images) is helpful but not required

Training Course Content

1. Docker-Based MySQL Lab Environment and Setup

- Overview of Docker-based training environment
- Why Docker for MySQL development
- Setting up MySQL containers
- Container initialization
- Connecting to MySQL
- Managing containers

Training Course Content

2. Advanced Query Patterns and Set-Based Thinking

- SQL review using lab environment
- Clean query structuring standards
- Set-based vs row-by-row processing
- Advanced filtering
- Conditional aggregation
- NULL handling and data quality

Training Course Content

3. Complex Joins and Multi-Table Query Design

- Join fundamentals in real systems
- JOIN filtering vs WHERE filtering
- Avoiding join explosion
- One-to-many / many-to-many joins
- Self-joins and hierarchy basics
- Anti-join patterns
- Reporting queries using multiple joins

Training Course Content

4. Subqueries and Common Table Expressions (CTEs)

- Subqueries vs joins (when and why)
- Correlated vs non-correlated subqueries
- Scalar subquery limitations and errors
- Derived tables and inline views
- CTEs using WITH
- Multi-step transformations using CTEs
- Recursive CTE basics
- Validating and debugging CTE queries

Training Course Content

5. Advanced Aggregations and Reporting SQL

- GROUP BY behavior in MySQL
- Aggregation strategies
- ORDER BY and LIMIT usage
- Top-N per group
- Latest record per group
- DISTINCT counting correctly
- Avoiding double-counting
- Date-based reporting

Training Course Content

6. Window Functions

- Window function fundamentals
- PARTITION BY vs ORDER BY
- Ranking
- Comparison
- Running totals and cumulative metrics
- Period comparisons
- Moving averages

Training Course Content

7. High Availability and Data Recovery

- High availability concepts
- MySQL replication using Docker
- Read scaling
- Failover basics (manual simulation)
- Backup strategies
- Point-in-time recovery
- Monitoring replication

Training Course Content

8. SQL Performance and Query Optimization

- Query execution flow
- Execution plan analysis
- Indexing strategies
- Common performance issues
- Optimization techniques

Training Course Content

9. MySQL Configuration and Performance Tuning

- Understanding MySQL configuration files
- Key performance parameters
- Tuning for workload types
- Resource management in Docker
- Monitoring performance
- Identifying bottlenecks

Training Course Content

10. Understanding MySQL Logs

- Types of logs
- Enabling and configuring logs
- Analyzing slow queries
- Using logs for troubleshooting
- Log rotation and management

Training Course Content

11. Avoiding Metadata Locks and Concurrency Issues

- What are metadata locks (MDL)
- Common causes
- Impact on production systems
- Detecting metadata locks
- Strategies to avoid blocking
- Safe schema changes in production

Training Course Content

12. MySQL Security and Encryption

- MySQL security fundamentals
- User and privilege management
- Encryption concepts
- Managing SSL connections
- Key management basics
- Securing backups and logs
- Best practices for production security

Training Course Content

13. Best Practices for Production-Ready SQL

- SQL coding standards
- Naming conventions
- Writing maintainable SQL
- Using views effectively
- Defensive SQL techniques
- Data quality checks and validation patterns

Copyright Protected

- Copyright © 2024 Cris Perando [crisperando@gmail.com]. All Rights Reserved.
- No part of this work may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the copyright holder, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.
- TrainOSys is a registered trademark of www.trainosys.com.
- All other trademarks mentioned herein are the property of their respective owners.

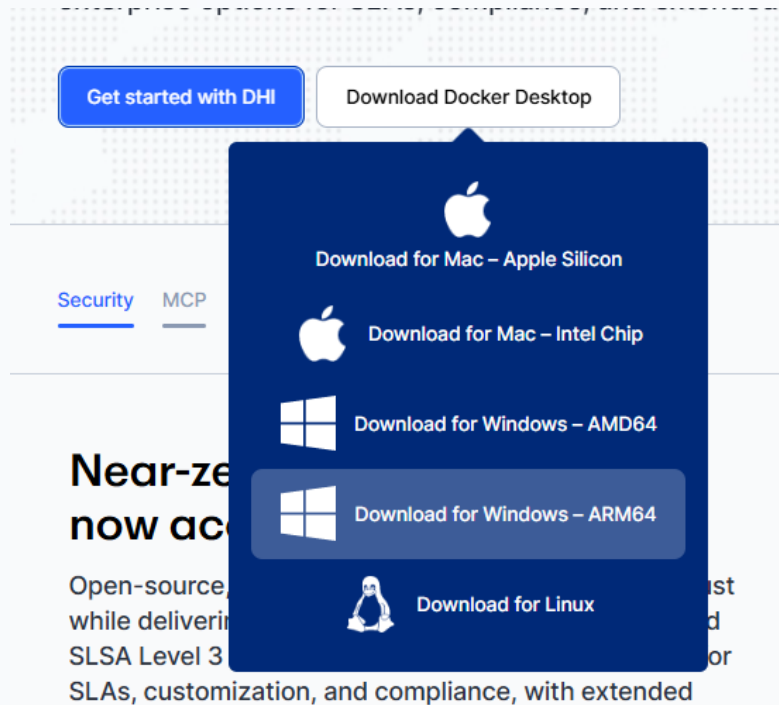
Docker-Based MySQL Lab Environment and Setup

Overview of Docker

- **Docker** is a containerization platform that allows you to run MySQL inside lightweight, isolated environments called **containers**.
- Each container behaves like a **mini server**.

Installing Docker

- Go to **docker.com** and **download** the installer and **install**



Why Docker for MySQL development

- Isolation
- Reproducibility
- Portability
- Fast Setup and Reset

Setting up MySQL containers

1. Pull the MySQL image

`docker pull mysql:8.4`

2. Run a MySQL container

➤ For multiline command

- Backtick (`) PowerShell
- Caret (^) CMD
- Backslash (\) Linux

Setting up MySQL containers

2. Run a MySQL container

```
docker run -d \  
--name mysql-training \  
-e MYSQL_ROOT_PASSWORD=Admin12345 \  
-e MYSQL_DATABASE=trainingdb \  
-e MYSQL_ROOT_HOST=% \  
-p 3306:3306 mysql:8.4
```



```
PS C:\Users\crisp> docker run -d --name mysql-training -e MYSQL_ROOT_PASSWORD=Admin12345 -e MYSQL_DATABASE=trainingdb -e
MYSQL_ROOT_HOST=% -p 3306:3306 mysql:8.4
000cf7ccbebbf9a5b5c4a8ac236ab24386204acd2a70c18f8882bc687de8f21b
PS C:\Users\crisp> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
000cf7ccbebb	mysql:8.4	"docker-entrypoint.s..."	18 seconds ago	Up 17 seconds	0.0.0.0:3306->3306/tcp, 33060/tcp
mysql-training					

Containers [Give feedback](#)

Container CPU usage ⓘ

0.76% / 3200% (32 CPUs available)

Container memory usage ⓘ

481.3MB / 15.14GB

[Show charts](#)



Only running

☐		Name	Container ID	Image	Port(s)	CPU (%)	Actions
☐	☐	oracle-db	527e8a8a6a0b	database/fi	1521:1521	0%	<input type="play"/> ⋮ <input type="trash"/>
☐	☐	postgres-contain	e86c5d643c5f	postgres:17	5433:5432	0%	<input type="play"/> ⋮ <input type="trash"/>
☐	☒	mysql-training	9bbd23126f4a	mysql:8.4	3306:3306 ↗	0.61%	<input checked="" type="play"/> ⋮ <input type="trash"/>



Setting up MySQL containers

3. Check container status

`docker ps`

4. View container logs

`docker logs mysql-training`

```
PowerShell
PS C:\Users\crisp> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               N
AMES
9bbd23126f4a   mysql:8.4  "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes  0.0.0.0:3306->3306/tcp, 33060/tcp  m
ysql-training
PS C:\Users\crisp> docker logs mysql-training
2026-04-14 22:32:59+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.8-1.el9 started.
2026-04-14 22:32:59+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-04-14 22:32:59+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.8-1.el9 started.
2026-04-14 22:32:59+00:00 [Note] [Entrypoint]: Initializing database files
2026-04-14T22:32:59.889278Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
2026-04-14T22:32:59.891618Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.4.8) initializing of server in pr
ogress as process 80
2026-04-14T22:32:59.912361Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-04-14T22:33:00.590584Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-04-14T22:33:03.846048Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Please c
onsider switching off the --initialize-insecure option.
2026-04-14T22:33:08.223264Z 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
2026-04-14 22:33:08+00:00 [Note] [Entrypoint]: Database files initialized
2026-04-14 22:33:08+00:00 [Note] [Entrypoint]: Starting temporary server
2026-04-14T22:33:08.352626Z 0 [System] [MY-015015] [Server] MySQL Server - start.
2026-04-14T22:33:08.622669Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.8) starting as process 136
2026-04-14T22:33:08.670489Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-04-14T22:33:09.317312Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-04-14T22:33:10.024413Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-04-14T22:33:10.024475Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted conn
ections are now supported for this channel.
2026-04-14T22:33:10.033392Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/m
ysqld' in the path is accessible to all OS users. Consider choosing a different directory.
2026-04-14T22:33:10.050990Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Socket: /var/run/mysqld/mysq
```



Setting up MySQL containers

5. Connect to MySQL inside the container

docker exec -it mysql-training mysql -uroot -p

```
PS C:\Users\crisp> docker exec -it mysql-training mysql -uroot -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.8 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Stop, start, and remove the container

`docker stop mysql-training`

`docker start mysql-training`

`docker restart mysql-training`

`docker rm -f mysql-training`

Stop, start, and remove the container

```
PowerShell
PS C:\Users\crisp> docker stop mysql-training
mysql-training
PS C:\Users\crisp> docker start mysql-training
mysql-training
PS C:\Users\crisp> docker restart mysql-training
mysql-training
PS C:\Users\crisp> docker rm -f mysql-training
mysql-training
PS C:\Users\crisp> docker ps
CONTAINER ID    IMAGE    COMMAND    CREATED    STATUS    PORTS    NAMES
PS C:\Users\crisp> |
```

Persistent storage with volumes

- By default, Docker containers are **ephemeral**.
- When you delete a container, all its data is lost.
- Volumes solve this by storing data **outside the container**, making it persistent.

Persistent storage with volumes

1. Create a Volume

```
docker volume rm mysql_training_data  
docker volume create mysql_training_data
```

2. Run MySQL with Volume

```
docker run -d \  
--name mysql-training \  
-e MYSQL_ROOT_PASSWORD=Admin12345 \  
-e MYSQL_DATABASE=trainingdb \  
-e MYSQL_ROOT_HOST=%  
-p 3306:3306 \  
-v mysql_training_data:/var/lib/mysql mysql:8.4
```

Persistent storage with volumes

```
PowerShell
PS C:\Users\crisp> docker volume create mysql_training_data
mysql_training_data
PS C:\Users\crisp> docker run -d --name mysql-training -e MYSQL_ROOT_PASSWORD=root123 -e MYSQL_DATABASE=trainingdb -p 3306:3306 -v mysql_training_data:/var/lib/mysql mysql:8.4
0902a24cfbed99bdd8e17f9661fb04fb6b7245e6e1c1595c1342a8943ecbd87e
PS C:\Users\crisp> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0902a24cfbed	mysql:8.4	"docker-entrypoint.s..."	4 seconds ago	Up 3 seconds	0.0.0.0:3306->3306/tcp, 33060/tcp	mysql-training

```
PS C:\Users\crisp> |
```

Advanced Query Patterns and Set-Based Thinking

Incorrect Joins (Common Pitfalls in SQL)

1. Missing JOIN Condition (Cartesian Product)

```
-- No ON condition, so every customer is matched with every order.  
SELECT *  
FROM customers c  
JOIN orders o;  
  
-- Corrected  
SELECT *  
FROM customers c  
JOIN orders o  
    ON c.customer_id = o.customer_id;
```

Incorrect Joins (Common Pitfalls in SQL)

2. Wrong JOIN Condition

```
-- Joining unrelated columns.  
SELECT *  
FROM orders o  
JOIN products p  
    ON o.order_id = p.product_id; -- wrong relationship  
  
-- Corrected  
SELECT *  
FROM orders o  
JOIN order_items oi  
    ON o.order_id = oi.order_id  
JOIN products p  
    ON oi.product_id = p.product_id;
```

Incorrect Joins (Common Pitfalls in SQL)

3. INNER JOIN Instead of LEFT JOIN (Data Loss)

-- Customers without orders are removed.

SELECT

c.first_name,

o.order_id

FROM customers c

INNER JOIN orders o

ON c.customer_id = o.customer_id;

-- Corrected

SELECT

c.first_name,

o.order_id

FROM customers c

LEFT JOIN orders o

ON c.customer_id = o.customer_id;



Incorrect Joins (Common Pitfalls in SQL)

4. Filtering in WHERE Breaking LEFT JOIN

```
-- WHERE removes NULL rows from the outer join result.  
-- LEFT JOIN behaves like INNER JOIN unintentionally.  
SELECT  
    c.customer_id,  
    c.first_name,  
    o.order_id,  
    o.order_date  
FROM customers c  
LEFT JOIN orders o  
    ON c.customer_id = o.customer_id  
WHERE o.order_date >= '2026-01-01';
```

Incorrect Joins (Common Pitfalls in SQL)

4. Filtering in WHERE Breaking LEFT JOIN

```
-- Corrected
SELECT
    c.customer_id,
    c.first_name,
    o.order_id,
    o.order_date
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
    AND o.order_date >= '2026-01-01';
```


Incorrect Joins (Common Pitfalls in SQL)

5. Duplicate rows caused by one-to-many joins

```
-- One order can have many order items
-- Each order is repeated once per item
-- If the goal is one row per order, this result is wrong
SELECT
    c.CustomerID,
    c.FirstName,
    o.OrderID
FROM Customers c
JOIN Orders o
    ON c.CustomerID = o.CustomerID
JOIN OrderItems oi
    ON o.OrderID = oi.OrderID;
```

Incorrect Joins (Common Pitfalls in SQL)

5. Duplicate rows caused by one-to-many joins

```
-- Corrected if the goal is one row per order
SELECT
    c.customer_id,
    c.first_name,
    o.order_id,
    COUNT(*) AS item_count
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY
    c.customer_id,
    c.first_name,
    o.order_id;
```



Incorrect Joins (Common Pitfalls in SQL)

5. Duplicate rows caused by one-to-many joins

```
-- Corrected if the goal is order detail rows
SELECT
    c.customer_id,
    c.first_name,
    o.order_id,
    oi.product_id,
    oi.quantity
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id;
```

Incorrect Joins (Common Pitfalls in SQL)

6. Using DISTINCT as a quick fix instead of solving the join problem

```
-- Bad: DISTINCT hides the duplication instead of fixing the logic.
```

```
SELECT DISTINCT  
    c.customer_id,  
    c.first_name,  
    o.order_id  
FROM customers c  
JOIN orders o  
    ON c.customer_id = o.customer_id  
JOIN order_items oi  
    ON o.order_id = oi.order_id;
```

Incorrect Joins (Common Pitfalls in SQL)

6. Using DISTINCT as a quick fix instead of solving the join problem

```
-- Correct if the goal is one row per order
SELECT
    c.customer_id,
    c.first_name,
    o.order_id
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id;
```

Clean query structuring standards

1. Write queries in logical blocks

```
SELECT  
FROM  
JOIN  
ON  
WHERE  
GROUP BY  
HAVING  
ORDER BY  
LIMIT;
```

Clean query structuring standards

2. Put each selected column on its own line

Good

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    c.email
FROM customers AS c;
```

Bad

```
SELECT c.customer_id, c.first_name,
c.last_name, c.email
FROM customers c;
```

Clean query structuring standards

3. Use consistent indentation

```
SELECT
    o.order_id,
    o.order_date,
    c.first_name,
    c.last_name
FROM orders o
INNER JOIN customers c
    ON c.customer_id = o.customer_id
WHERE o.status = 'Completed';
```


Clean query structuring standards

4. Use table aliases carefully

Good

```
SELECT
    o.order_id,
    c.first_name,
    p.product_name
FROM orders o
JOIN customers c
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON oi.order_id = o.order_id
JOIN products p
    ON p.product_id = oi.product_id;
```

Bad

```
SELECT *
FROM orders a
JOIN customers b
    ON b.customer_id = a.customer_id;
```

Clean query structuring standards

5. Always qualify columns in joins

Good

```
SELECT
    o.order_id,
    c.customer_id
FROM orders o
JOIN customers c
    ON c.customer_id = o.customer_id;
```

Bad

```
-- Ambiguous and incorrect join
condition
SELECT
    order_id,
    customer_id
FROM orders
JOIN customers
    ON customer_id = customer_id;
```

Clean query structuring standards

6. Avoid SELECT *

Good

```
SELECT
    customer_id,
    first_name,
    last_name,
    email
FROM customers;
```

Bad

```
SELECT *
FROM customers;
```

Clean query structuring standards

7. Separate join conditions from filter conditions

```
SELECT
    o.order_id,
    c.first_name
FROM orders o
JOIN customers c
    ON c.customer_id = o.customer_id
WHERE o.status = 'Completed';
```

Clean query structuring standards

8. Format complex conditions vertically

For multiple predicates, place each condition on its own line.

```
SELECT
    order_id,
    customer_id,
    status
FROM orders
WHERE status = 'Completed'
    AND order_date >= '2026-01-01'
    AND customer_id IS NOT NULL;
```

For mixed AND/OR logic, use parentheses clearly

```
SELECT
    order_id,
    status
FROM orders
WHERE (
    status = 'Pending'
    OR status = 'Processing'
)
    AND order_date >= '2026-01-01';
```

Clean query structuring standards

9. Use uppercase for SQL keywords

```
SELECT
    product_name,
    price
FROM products
WHERE price > 1000
ORDER BY price DESC;
```

Clean query structuring standards

10. Use meaningful derived column names

```
SELECT
    o.order_id,
    SUM(oi.quantity * oi.unit_price) AS order_total
FROM orders o
JOIN order_items oi
    ON oi.order_id = o.order_id
GROUP BY o.order_id;
```

Clean query structuring standards

11. Keep expressions readable

```
SELECT
    oi.order_id,
    oi.quantity,
    oi.unit_price,
    oi.quantity * oi.unit_price AS line_total
FROM order_items oi;
```


Clean query structuring standards

12. Group related columns together

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    c.city,
    COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o
    ON o.customer_id = c.customer_id
GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name,
    c.city;
```



Clean query structuring standards

13. Keep GROUP BY formatted and explicit

```
GROUP BY  
    c.customer_id,  
    c.first_name,  
    c.last_name,  
    c.city;
```

Clean query structuring standards

14. Avoid deeply nested SQL when a clearer structure exists

```
WITH order_totals AS (  
    SELECT  
        oi.order_id,  
        SUM(oi.quantity * oi.unit_price) AS order_total  
    FROM order_items oi  
    GROUP BY oi.order_id  
)  
SELECT  
    o.order_id,  
    c.first_name,  
    c.last_name,  
    ot.order_total  
FROM orders o  
JOIN customers c  
    ON c.customer_id = o.customer_id  
JOIN order_totals ot  
    ON ot.order_id = o.order_id;
```

Clean query structuring standards

15. Comment only when needed

```
-- Helpful: Only include completed orders for revenue reporting
SELECT
    order_id,
    order_date
FROM orders
WHERE status = 'Completed';

-- Not helpful: Selecting only order_id (no business value)
SELECT
    order_id
FROM orders;
```

Set-based vs row-by-row processing

- Set-based processing means you write SQL to work on a group of rows at once.
- Row-by-row processing means handling one row at a time, usually with loops, cursors, or procedural logic.

Example of Set-based

This updates all matching rows in one operation.

```
UPDATE orders
SET status = 'Archived'
WHERE status = 'Shipped'
      AND order_date < '2026-01-01';
```

Row-by-row

- **Row-by-row processing** is done using a **cursor inside a stored procedure**.
- Row-by-row processing is often slower because it repeatedly executes logic for each row.
- A common phrase for bad row-by-row SQL is **Row By Agonizing Row (RBAR)**

Example of Row-by-row

This processes records one at a time.

```
DELIMITER $$

CREATE PROCEDURE update_orders_row_by_row()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_order_id BIGINT;

    -- Cursor definition
    DECLARE order_cursor CURSOR FOR
        SELECT order_id
        FROM orders
        WHERE status = 'Shipped'
        AND order_date < '2026-01-01';
```


Example of Row-by-row

This processes records one at a time.

```
-- Handler for end of cursor
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

-- Open cursor
OPEN order_cursor;

read_loop: LOOP
    FETCH order_cursor INTO v_order_id;

    IF done = 1 THEN
        LEAVE read_loop;
    END IF;
```

Example of Row-by-row

This processes records one at a time.

```
-- Row-by-row update
UPDATE orders
SET status = 'Archived'
WHERE order_id = v_order_id;

END LOOP;

-- Close cursor
CLOSE order_cursor;
END$$

DELIMITER ;

CALL update_orders_row_by_row();
```



When row-by-row processing is VALID

1. Complex per-row business logic

```
IF customer_type = 'VIP' THEN
    -- special logic
ELSE
    -- normal logic
END IF;
```

When row-by-row processing is VALID

2. Calling external operations per row

- When each row must trigger something **outside SQL**
 - API calls
 - sending emails
 - logging to external systems

When row-by-row processing is VALID

5. Data migration / ETL edge cases

➤ When transforming data with

- row-specific validation
- error handling per record
- logging failures per row

When row-by-row processing is VALID

6. When required by procedural workflows

- stored procedures
- workflows with multiple steps
- retry logic per row

Advanced filtering

1. **CASE in SELECT** - Used when you want to **classify, label, or transform output**.

```
SELECT
    order_id,
    status,
    CASE
        WHEN status = 'Shipped' THEN 'Completed'
        WHEN status = 'Pending' THEN 'In Progress'
        WHEN status = 'Cancelled' THEN 'Closed'
        ELSE 'Other'
    END AS status_group
FROM orders;
```

Advanced filtering

2. CASE in WHERE - **This should used carefully** because it can make queries harder to read and may hurt performance.

```
SELECT
    order_id,
    customer_id,
    order_date,
    status
FROM orders
WHERE CASE
    WHEN status = 'Shipped' THEN 1
    WHEN status = 'Pending' THEN 1
    ELSE 0
    END = 1;
```

```
-- Cleaner version

SELECT
    order_id,
    customer_id,
    order_date,
    status
FROM orders
WHERE status IN ('Shipped',
    'Pending');
```



Advanced filtering

3. Problematic CASE in WHERE

```
-- Non-sargable:  
-- the predicate wraps business logic around  
-- filtering,  
-- so the optimizer may not use an index  
-- efficiently.  
SELECT  
    order_id,  
    customer_id,  
    order_date,  
    status  
FROM orders  
WHERE CASE  
    WHEN status = 'Shipped' THEN order_date  
    ELSE '1900-01-01'  
END >= '2026-01-01';
```

```
-- Corrected  
SELECT  
    order_id,  
    customer_id,  
    order_date,  
    status  
FROM orders  
WHERE status = 'Shipped'  
    AND order_date >= '2026-01-01';
```



Advanced filtering

3. Problematic CASE in WHERE

```
-- Evaluated row-by-row  
-- Can rewrite into: WHERE Status IN ('Shipped', 'Pending')
```

```
SELECT  
    order_id,  
    customer_id,  
    order_date,  
    status  
FROM orders  
WHERE CASE  
    WHEN status = 'Shipped' THEN 1  
    WHEN status = 'Pending' THEN 1  
    ELSE 0  
    END = 1;
```

Advanced filtering

4. Avoiding non-sargable filtering

```
-- Index on order_date may not be  
used efficiently  
-- because the column is wrapped  
in YEAR().
```

```
SELECT  
    order_id,  
    customer_id,  
    order_date,  
    status  
FROM orders  
WHERE YEAR(order_date) = 2026;
```

```
-- Corrected
```

```
SELECT  
    order_id,  
    customer_id,  
    order_date,  
    status  
FROM orders  
WHERE order_date >= '2026-01-01'  
    AND order_date < '2027-01-01';
```

Advanced filtering

5. Filtering on computed expressions

-- Keep the column bare on the left side for better optimization

-- Avoid applying arithmetic directly to the indexed column when possible.

SELECT

product_id,
product_name,
price

FROM products

WHERE price * 1.12 > 1000;

-- Corrected

SELECT

product_id,
product_name,
price

FROM products

WHERE price > 1000 / 1.12;



Conditional aggregation

- Conditional aggregation means **aggregating only the rows that match a condition** inside the aggregate function.
 - `sum(case when condition then value else 0 end)`
- This is very useful for
 - Counting by category
 - Summing only specific rows
 - Building multiple metrics in one query

Conditional aggregation

```
-- Count the Total Orders, Shipped, Pending and Cancelled
select
    count(*) as total_orders,
    sum(case when status = 'shipped' then 1 else 0 end) as shipped_orders,
    sum(case when status = 'pending' then 1 else 0 end) as pending_orders,
    sum(case when status = 'cancelled' then 1 else 0 end) as
cancelled_orders
from orders;
```

orders (1r x 4c)				
#	1 total_orders	2 shipped_orders	3 pending_orders	4 cancelled_orders
1	10,000	2,509	2,497	2,510



Conditional aggregation

```
-- Count orders by status group by customer_id
select
  customer_id,
  count(*) as total_orders,
  sum(case when status = 'shipped' then 1 else 0 end) as shipped_orders,
  sum(case when status = 'pending' then 1 else 0 end) as pending_orders,
  sum(case when status = 'cancelled' then 1 else 0 end) as cancelled_orders
from orders
group by customer_id;
```

orders (1,000r x 5c)						
#	1 customer_id	2 total_orders	3 shipped_orders	4 pending_orders	5 cancelled_orders	
1	1	9	0	4	2	
2	2	12	4	4	3	
3	3	4	2	0	1	
4	4	9	3	3	2	
5	5	15	2	3	2	

Conditional aggregation

```
-- Get the sum sales only for shipped orders
select
    o.customer_id,
    sum(case when o.status = 'shipped' then oi.quantity * oi.unit_price else
0 end) as shipped_sales
from orders o
join order_items oi
    on oi.order_id = o.order_id
group by o.customer_id;
```

orders (1,000r x 2c)		
#	1 customer_id	2 shipped_sales
1	257	29,907.69
2	970	18,184.93



Activity 1 - Conditional Aggregation

- Create a conditional aggregation that will generate the following report
 - Total Orders
 - Shipped Orders
 - Pending Orders
 - Shipped Revenue
 - Pending Value

Activity 1 - Conditional Aggregation

orders (1,000r × 6c)							
#	1 customer_id	 2 total_orders	3 shipped_order_rows	4 pending_order_rows	5 shipped_revenue	6 pending_value	
1	1	9	0	12	0.0	20,437.6	
2	2	12	12	12	13,674.74	11,950.41	
3	3	4	6	0	13,392.61	0.0	
4	4	9	9	9	11,257.53	16,883.17	
5	5	15	6	9	8,564.78	15,112.45	
6	6	10	9	3	11,043.56	2,273.98	
7	7	8	3	12	2,236.15	21,003.45	
8	8	13	12	9	21,333.05	14,724.41	
-	-	-	-	-	-	-	-

NULL handling and data quality

- **NULL** means **missing / unknown / not applicable**.
- It is not
 - **0**
 - **"** empty string
 - **'N/A'**
 - **'0000-00-00'**

NULL handling and data quality

- Therefore, it is safe to say that

```
mysql> SELECT 1 = NULL;
+-----+
| NULL |
+-----+
mysql> SELECT NULL = NULL;
+-----+
| NULL |
+-----+
mysql> SELECT NULL IS NULL;
+-----+
| 1 |
+-----+
```

NULL handling and data quality

- Ignoring NULL in calculations

```
-- If price is NULL, result is also NULL.  
SELECT product_name, price * 0.10 AS discount_amount  
FROM products;  
  
-- Safer  
SELECT product_name, IFNULL(price, 0) * 0.10 AS discount_amount  
FROM products;
```

NULL handling and data quality

- NULL breaking string concatenation

```
-- If one part is NULL, CONCAT() returns
NULL.
SELECT CONCAT(first_name, ' ', last_name)
AS full_name
FROM customers;

-- Safer
SELECT CONCAT(IFNULL(first_name, ''), ' ',
IFNULL(last_name, '')) AS full_name
FROM customers;
```

#	1 full_name
994	first994 last994
995	first995 last995
996	first996 last996
997	first997 last997
998	first998 last998
999	first999 last999
1000	last1000

NULL handling and data quality

- MySQL functions for NULL handling

```
-- Returns alt_value if expr is NULL.  
-- IFNULL(expr, alt_value)
```

```
SELECT product_name, IFNULL(price, 0) AS safe_price  
FROM products;
```

NULL handling and data quality

- MySQL functions for NULL handling

```
-- COALESCE(val1, val2, val3, ...)  
-- Returns the first non-NULL value.  
SELECT  
    customer_id,  
    COALESCE(city, 'Unknown') AS city_label  
FROM customers;  
  
-- More flexible than IFNULL():  
SELECT  
    COALESCE(NULL, NULL, 'Fallback') AS result;
```


NULL handling and data quality

- MySQL functions for NULL handling

```
-- COALESCE(val1, val2, val3, ...)
-- Returns the first non-NULL value.
SELECT
    customer_id,
    COALESCE(city, 'Unknown') AS city_label
FROM customers;

-- More flexible than IFNULL():
SELECT
    COALESCE(NULL, NULL, 'Fallback') AS result;
```

NULL handling and data quality

- MySQL functions for NULL handling

```
-- NULLIF(expr1, expr2)
-- Returns NULL if expr1 = expr2, otherwise returns expr1.
-- Useful for treating bad placeholder values as NULL:
```

```
SELECT
    customer_id,
    NULLIF(email, '') AS normalized_email
FROM customers;
```

```
-- Also useful to avoid divide-by-zero
SELECT
    product_id,
    price / NULLIF(0, 0) AS price_per_unit
FROM products;
```



NULL handling and data quality

- Data quality cleanup patterns

```
-- Data quality cleanup patterns
-- normalize city values
SELECT
    customer_id,
    COALESCE(NULLIF(TRIM(city), ''), 'Unknown') AS clean_city
FROM customers;
```

customers (1,000r x 2c)			
#	1 customer_id		2 clean_city
65	975		Unknown
66	990		Unknown
67	4		Bacolod



Null Affect Joins

-- If no order exists, o.order_id becomes NULL. Normal in Left Join

SELECT

c.customer_id,
c.first_name,
o.order_id

FROM customers c

LEFT JOIN orders o

ON c.customer_id = o.customer_id;

-- The Left Join become Inner Join because of Nullity Test

SELECT

c.customer_id,
c.first_name,
o.order_id

FROM customers c

LEFT JOIN orders o

ON c.customer_id = o.customer_id

WHERE o.order_id **IS NOT NULL**;



Null Affect Aggregates

```
-- COUNT(*) counts all rows
-- COUNT(email) counts only non-NULL emails
SELECT
    COUNT(*) AS total_rows,
    COUNT(email) AS rows_with_email
FROM customers;

-- AVG() ignores NULL prices.
SELECT
    AVG(price) AS avg_price
FROM products;

-- Replace NULL with 0, result changes:
SELECT
    AVG(IFNULL(price, 0)) AS avg_price_with_zero
FROM products;
```



Conditional aggregation with NULL-safe

```
-- Conditional aggregation with NULL-safe logic
SELECT
    SUM(CASE WHEN email IS NOT NULL AND TRIM(email) <> '' THEN 1 ELSE 0
END) AS with_email,
    SUM(CASE WHEN email IS NULL OR TRIM(email) = '' THEN 1 ELSE 0 END) AS
missing_email
FROM customers;

-- Order counts by status with NULL-safe fallback
SELECT
    SUM(CASE WHEN status = 'Shipped' THEN 1 ELSE 0 END) AS shipped_orders,
    SUM(CASE WHEN status = 'Pending' THEN 1 ELSE 0 END) AS pending_orders,
    SUM(CASE WHEN status IS NULL THEN 1 ELSE 0 END) AS unknown_status
FROM orders;
```



Complex Joins and Multi-Table Query Design

Avoiding join explosion and duplicate rows

- Distinct should be use in distinct data not to hide the problem
- One customer can have many orders
- One order can have many order items
- Result expands naturally across matching rows

Avoiding join explosion and duplicate rows

-- Issue: row multiplication

SELECT

c.customer_id,
c.first_name,
o.order_id,
oi.order_item_id

FROM customers c

JOIN orders o

ON c.customer_id = o.customer_id

JOIN order_items oi

ON o.order_id = oi.order_id;

Avoiding join explosion and duplicate rows

```
-- Corrected: Safer reporting pattern
SELECT
    c.customer_id,
    c.first_name,
    COUNT(DISTINCT o.order_id) AS order_count,
    SUM(oi.quantity * oi.unit_price) AS total_sales
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY
    c.customer_id,
    c.first_name;
```

Avoiding join explosion and duplicate rows

customers (975r × 4c)				
#	1 customer_id	 2 first_name	3 order_count	4 total_sales
1	2	first2	10	1,389.32
2	3	first3	10	1,262.12
3	4	first4	10	1,118.52
4	5	first5	10	1,075.04
5	6	first6	10	1,780.9
6	7	first7	10	2,487.7
7	8	first8	10	1,969.2
8	9	first9	10	1,660.0
9	10	first10	10	1,746.6
10	11	first11	10	2,570.8
11	12	first12	10	3,682.52
12	13	first13	10	2,772.72

One-to-many and many-to-many joins

```
-- orders → order_items = one-to-many
-- orders ↔ products through order_items = many-to-many
SELECT
    o.order_id,
    p.product_id,
    p.product_name,
    oi.quantity,
    oi.unit_price
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
JOIN products p
    ON oi.product_id = p.product_id
```

One-to-many and many-to-many joins

Result #1 (29,334r × 5c)						
#	1 order_id	2 product_id	3 product_name	4 quantity	5 unit_price	
1	497	1	product1	1	456.81	
2	1,497	1	product1	1	456.81	
3	2,497	1	product1	1	456.81	
4	3,497	1	product1	1	456.81	
5	5,497	1	product1	1	456.81	
6	6,497	1	product1	1	456.81	
7	7,497	1	product1	1	456.81	
8	8,497	1	product1	1	456.81	
9	9,497	1	product1	1	456.81	
10	498	1	product1	1	916.36	
11	1,498	1	product1	1	916.36	
12	2,498	1	product1	1	916.36	
13	3,498	1	product1	1	916.36	

Multi-table reporting joins

```
-- Sales Detail Reports
SELECT
    o.order_id,
    o.order_date,
    c.first_name,
    c.last_name,
    p.product_name,
    oi.quantity,
    oi.unit_price,
    oi.quantity * oi.unit_price AS line_total
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
JOIN products p
    ON oi.product_id = p.product_id
ORDER BY
    o.order_id,
    p.product_name;
```



Multi-table reporting joins

Result #1 (28,584r × 8c)								
#	1 order_id	2 order_date	3 first_name	4 last_name	5 product_name	6 quantity	7 unit_price	8 line_total
1	1	2026-04-14 10:34:44	first2	last2	product3	3	10.91	32.73
2	1	2026-04-14 10:34:44	first2	last2	product4	4	11.82	47.28
3	1	2026-04-14 10:34:44	first2	last2	product5	5	12.73	63.65
4	2	2026-04-13 10:34:44	first3	last3	product4	4	11.82	47.28
5	2	2026-04-13 10:34:44	first3	last3	product5	5	13.64	68.2
6	2	2026-04-13 10:34:44	first3	last3	product6	1	15.46	15.46
7	3	2026-04-12 10:34:44	first4	last4	product5	5	12.73	63.65
8	3	2026-04-12 10:34:44	first4	last4	product6	1	15.46	15.46
9	3	2026-04-12 10:34:44	first4	last4	product7	2	18.19	36.38
10	4	2026-04-11 10:34:44	first5	last5	product6	1	13.64	13.64
11	4	2026-04-11 10:34:44	first5	last5	product7	2	17.28	34.56
12	4	2026-04-11 10:34:44	first5	last5	product8	3	20.92	62.76
13	5	2026-04-10 10:34:44	first6	last6	product7	2	14.55	29.1

Self-joins

```
-- Customers from the same city  
-- < is use to avoids matching the same row to itself and avoids mirrored  
duplicates.
```

SELECT

```
    c1.customer_id AS customer_1_id,  
    c1.first_name AS customer_1_name,  
    c2.customer_id AS customer_2_id,  
    c2.first_name AS customer_2_name,  
    c1.city
```

FROM customers c1

JOIN customers c2

```
    ON c1.city = c2.city
```

```
    AND c1.customer_id < c2.customer_id;
```



Anti-join patterns

```
-- Using NOT EXISTS
SELECT
    c.customer_id,
    c.first_name,
    c.last_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Anti-join patterns

```
-- Using LEFT JOIN ... IS NULL
SELECT
    c.customer_id,
    c.first_name,
    c.last_name
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Semi-join patterns

```
-- Semi-join patterns
SELECT
    c.customer_id,
    c.first_name,
    c.last_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Joining with aggregated subqueries

```
-- Joining with aggregated subqueries
SELECT
    c.customer_id,
    c.first_name,
    x.order_count,
    x.total_sales
FROM customers c
JOIN (
    SELECT
        o.customer_id,
        COUNT(DISTINCT o.order_id) AS order_count,
        SUM(oi.quantity * oi.unit_price) AS total_sales
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY o.customer_id
) x
ON c.customer_id = x.customer_id;
```



Joining with NULL awareness

```
-- Filtering after outer joins must be done carefully
SELECT
    o.order_id,
    o.customer_id,
    c.first_name
FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id;
```

Subqueries and Common Table Expressions (CTEs)

Subqueries vs joins (when and why)

- Use JOIN for **data shaping**, for example
 - JOIN multiple tables in the final output.
- Use subqueries for **decision making**, for example
 - Check existence or filter

Activity 2 - Subqueries vs Joins

- This activity will create a report of the customer last order occurrence

#	1 customer_id	2 first_name	3 last_order_date
1	1	first1	(NULL)
2	2	first2	2026-04-14 10:34:44
3	3	first3	2026-04-13 10:34:44
4	4	first4	2026-04-12 10:34:44
5	5	first5	2026-04-11 10:34:44
6	6	first6	2026-04-10 10:34:44
7	7	first7	2026-04-09 10:34:44
8	8	first8	2026-04-08 10:34:44
9	9	first9	2026-04-07 10:34:44
10	10	first10	2026-04-06 10:34:44
11	11	first11	2026-04-05 10:34:44
12	12	first12	2026-04-04 10:34:44
13	13	first13	2026-04-03 10:34:44



Activity 2 - Subqueries vs Joins

- *Correlated Subquery (Best fit for this scenario)*
- ✓ *Executes once per customer*
- ✓ *Uses index for targeted lookup*
- ✓ *Avoids building large intermediate result sets*
- ✓ *Very readable for "one value per parent row" problems*

SELECT

```
c.customer_id,  
c.first_name,  
(  
    SELECT MAX(o.order_date)  
    FROM orders o  
    WHERE o.customer_id = c.customer_id  
) AS last_order_date
```

FROM customers c;

Activity 2 - Subqueries vs Joins

```
-- JOIN with GROUP BY (Heavier for this scenario)
-- ✗ Expands rows (customer × orders)
-- ✗ Builds larger intermediate dataset
-- ✗ Requires grouping to collapse rows again
-- ✓ Better when multiple aggregates are needed
```

SELECT

```
    c.customer_id,
    c.first_name,
    MAX(o.order_date) AS last_order_date
```

FROM customers c

LEFT JOIN orders o

```
    ON o.customer_id = c.customer_id
```

GROUP BY

```
    c.customer_id,
    c.first_name;
```

Activity 2 - Subqueries vs Joins

- Execution Plan Comparison (Based on **EXPLAIN ANALYZE**)

Verdict	Correlated Subquery	JOIN + GROUP BY
1. Execution Pattern	Targeted lookup per customer	Expand rows then aggregate
2. Intermediate Rows	✗ No row expansion	⚠ ~9775 rows generated
3. Aggregation Strategy	Per-row (inside subquery)	Global (after join)
4. Performance	✓ Faster (~0.01 sec)	✗ Slower (~0.04 sec)
5. Best Use Case	Single value per row	Multiple aggregates / reporting

Correlated vs non-correlated subqueries

- **Non-correlated subquery**

- Runs once, independent of outer query
- Often optimized into semi-join
- Usually faster
- Use in filtering sets

- **Correlated subquery**

- Runs once per row of outer query
- Very slow if not carefully implemented
- Hard to optimize and it use to process row-by-row lookup

Example: Non-correlated subquery

```
-- Customers who have orders
SELECT *
FROM customers
WHERE customer_id IN (
    SELECT DISTINCT customer_id
    FROM orders
);
```

Example: Non-correlated subquery

```
-- Get last order date per customer
SELECT
    c.customer_id,
    c.first_name,
    (
        SELECT MAX(o.order_date)
        FROM orders o
        WHERE o.customer_id = c.customer_id
    ) AS last_order_date
FROM customers c;
```

Scalar subquery limitations and errors

1. Must Return ONLY ONE ROW

```
-- ✗ Error Case (Multiple Rows Returned)
SELECT
    c.customer_id,
    (
        SELECT o.order_date
        FROM orders o
        WHERE o.customer_id = c.customer_id
    ) AS order_date
FROM customers c;
```

Scalar subquery limitations and errors

2. Must Return ONLY ONE COLUMN

```
-- ✗ Error Case (Multiple Columns)
SELECT
c.customer_id,
(
    SELECT
    o.order_id,
    MAX(o.order_date)
    FROM orders o
    WHERE o.customer_id = c.customer_id
    LIMIT 1
)
FROM customers c;
```



Scalar subquery limitations and errors

2. Must Return ONLY ONE COLUMN

```
-- Corrected, return only ONE COLUMN
SELECT
    c.customer_id,
    (
        SELECT o.order_id
        FROM orders o
        WHERE o.customer_id = c.customer_id
        ORDER BY o.order_date DESC
        LIMIT 1
    ) AS latest_order_id
FROM customers c;
```

Scalar subquery limitations and errors

3. NULL Result Behavior (Silent Issue)

```
-- If no row is returned → result is NULL, not error.  
-- Fix, replace: WHERE COALESCE(t.last_order_date, '1900-01-01') > '2026-01-01';  
  
SELECT *  
FROM (  
    SELECT  
        c.customer_id,  
        (  
            SELECT MAX(o.order_date)  
            FROM orders o  
            WHERE o.customer_id = c.customer_id  
        ) AS last_order_date  
    FROM customers c  
    ) t  
WHERE t.last_order_date > '2026-01-01' -- NULL fails comparison
```

Scalar subquery limitations and errors

4. Performance Issue (Executed Per Row)

```
-- Runs once per product: With 1,000 customers → subquery runs 1,000 times
SELECT
    p.product_id,
    p.product_name,
    (
        SELECT SUM(oi.quantity)
        FROM order_items oi
        WHERE oi.product_id = p.product_id
    ) AS total_qty_sold
FROM products p;
```

Derived tables and inline views

- Derived tables and inline views are essentially the same.
 - SELECT statement written inside the FROM clause
 - Treated like a temporary result set
 - Given an alias so the outer query can use it
- It is useful for
 - Break a complex query into steps
 - Aggregate first, then filter or join later
 - Make the query more readable

Derived tables and inline views

- Basic syntax

```
SELECT ...  
FROM (  
    SELECT ...  
) AS dt;
```

Example - Basic derived tables

-- Suppose you want customers whose total order count is greater than 5.

```
SELECT
    x.customer_id,
    x.order_count
FROM (
    SELECT
        o.customer_id,
        COUNT(*) AS order_count
    FROM orders o
    GROUP BY o.customer_id
) AS x
WHERE x.order_count > 5;
```

Example - Basic derived tables

#	1 customer_id	2 order_count
1	(NULL)	250
2	2	10
3	3	10
4	4	10
5	5	10
6	6	10
7	7	10
8	8	10
9	9	10
10	10	10
11	11	10
12	12	10
13	13	10

Derived tables aggregate first, then join

-- Show customers with their total number of orders.

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    x.order_count
FROM customers c
LEFT JOIN (
    SELECT
        o.customer_id,
        COUNT(*) AS order_count
    FROM orders o
    GROUP BY o.customer_id
) AS x
ON x.customer_id = c.customer_id;
```


Derived tables aggregate first, then join

	1 customer_id		2 first_name	3 last_name	4 order_count
1	1		first1	last1	(NULL)
2	2		first2	last2	10
3	3		first3	last3	10
4	4		first4	last4	10
5	5		first5	last5	10
6	6		first6	last6	10
7	7		first7	last7	10
8	8		first8	last8	10
9	9		first9	last9	10
10	10		first10	last10	10
11	11		first11	last11	10
12	12		first12	last12	10
13	13		first13	last13	10

Derived tables aggregate first, then filter

```
-- Inner query computes summary
-- Outer query applies filter
SELECT
    s.order_id,
    s.order_total
FROM (
    SELECT
        oi.order_id,
        SUM(oi.quantity * oi.unit_price) AS order_total
    FROM order_items oi
    GROUP BY oi.order_id
) AS s
WHERE s.order_total >= 1000;
```

Having vs Where in Derived Tables

- **HAVING** when filtering aggregated results
- **WHERE** when filtering the raw rows and no aggregation involved
- *Always prefer WHERE when possible*
 - Uses indexes
 - Reduces data early
 - Faster performance

Having vs Where in Derived Tables

```
-- HAVING version of aggregated result
SELECT
    dt.order_id,
    dt.order_total
FROM (
    SELECT
        oi.order_id,
        SUM(oi.quantity * oi.unit_price) AS order_total
    FROM order_items oi
    GROUP BY oi.order_id
    HAVING SUM(oi.quantity * oi.unit_price) >= 1000
) AS dt;
```

Having vs Where in Derived Tables

```
-- WHERE version of raw row results
SELECT
    dt.order_id,
    dt.order_total
FROM (
    SELECT
        oi.order_id,
        SUM(oi.quantity * oi.unit_price) AS order_total
    FROM order_items oi
    GROUP BY oi.order_id
) AS dt
WHERE dt.order_total >= 1000;
```

CTEs using WITH

- **CTE** is a temporary result set at the start of a query using WITH.
- A named subquery that you can reuse inside your main query
- Why this is better than subquery
 - Cleaner structure
 - Easier to debug
 - Reusable

When to use CTE

- Complex query (multi-step logic)
- You need readability
- Reuse same result multiple times
- Recursive logic
- Avoid when:
 - Simple query (adds overhead)
 - Very large dataset used once (materialization cost)

CTEs using WITH

- General Syntax

```
WITH cte_name AS (  
    SELECT ...  
)  
SELECT *  
FROM cte_name;
```


CTE (Readable Alternative to Derived Table)

```
-- Get orders with total ≥ 1000
WITH order_totals AS (
    SELECT
        oi.order_id,
        SUM(oi.quantity * oi.unit_price) AS total_amount
    FROM order_items oi
    GROUP BY oi.order_id
)
SELECT *
FROM order_totals
WHERE total_amount >= 1000;
```

Multiple CTEs (Step-by-Step Query)

```
-- Find high-value customers
WITH order_totals AS (
    SELECT
        o.customer_id,
        SUM(oi.quantity * oi.unit_price) AS total_spent
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    GROUP BY o.customer_id
),
high_value_customers AS (
    SELECT *
    FROM order_totals
    WHERE total_spent >= 5000
)
```

Multiple CTEs (Step-by-Step Query)

```
SELECT
    c.customer_id,
    c.first_name,
    hvc.total_spent
FROM customers c
JOIN high_value_customers hvc
    ON c.customer_id = hvc.customer_id;
```

Reusing CTE

```
-- You compute once, reuse many times
WITH order_totals AS (
    SELECT
        o.customer_id,
        SUM(oi.quantity * oi.unit_price) AS total_spent
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    GROUP BY o.customer_id
)
SELECT
    AVG(total_spent) AS avg_spent,
    MAX(total_spent) AS max_spent
FROM order_totals;
```

Reusing CTE

```
-- You compute once, reuse many times
WITH order_totals AS (
    SELECT
        o.customer_id,
        SUM(oi.quantity * oi.unit_price) AS total_spent
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    GROUP BY o.customer_id
)
SELECT
    AVG(total_spent) AS avg_spent,
    MAX(total_spent) AS max_spent
FROM order_totals;
```

What is a Recursive CTE

- Recursive CTE query that refers to itself to build a result step-by-step.

```
WITH RECURSIVE cte_name AS (  
    -- 1. Anchor (starting point)  
    SELECT ...  
  
    UNION ALL  
  
    -- 2. Recursive part (refers to itself)  
    SELECT ...  
    FROM cte_name  
    WHERE ...  
)  
SELECT * FROM cte_name;
```

Recursive CTE

```
-- Generate numbers 1 to 10
WITH RECURSIVE numbers_cte AS (
    SELECT 1 AS n

    UNION ALL

    SELECT n + 1
    FROM numbers_cte
    WHERE n < 10
)
SELECT * FROM numbers_cte;
```

Recursive CTE

```
numbers_cte (10r × 1c)
```

	1 n
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10

Recursive CTE

```
-- Date range recursion
-- Date range + orders report
WITH RECURSIVE date_range AS (
    SELECT DATE('2026-01-01') AS order_day

    UNION ALL

    SELECT order_day + INTERVAL 1 DAY
    FROM date_range
    WHERE order_day < '2026-01-10'
)
```

Recursive CTE

```
SELECT
    dr.order_day,
    COUNT(o.order_id) AS order_count
FROM date_range dr
LEFT JOIN orders o
    ON o.order_date >= dr.order_day
    AND o.order_date < dr.order_day + INTERVAL 1 DAY
GROUP BY dr.order_day
ORDER BY dr.order_day;
```

Recursive CTE

date_range (10r × 2c)		
	1 order_day	2 order_count
1	2026-01-01	28
2	2026-01-02	28
3	2026-01-03	28
4	2026-01-04	28
5	2026-01-05	24
6	2026-01-06	28
7	2026-01-07	28
8	2026-01-08	28
9	2026-01-09	28
10	2026-01-10	24

Validating and debugging CTE queries

- WITH must come before the main query
- Every CTE needs a name
- CTE query must be enclosed in parentheses
- If you define multiple CTEs, separate them with commas
- The main statement must immediately follow the CTE

Common CTE debugging checklist

- Is the syntax correct?
- Does the inner query work by itself?
- Are the output columns correct?
- Are aliases spelled correctly?
- Is the grouping correct?
- Are filters placed in the right stage?
- For recursive CTEs, is there a stopping condition?
- Are row counts reasonable?
- Does EXPLAIN ANALYZE show unexpected scanning or materialization?



Test the CTE logic separately

- Instead of:

```
WITH order_totals AS (  
    SELECT  
        oi.order_id,  
        SUM(oi.quantity * oi.unit_price) AS total_amount  
    FROM order_items oi  
    GROUP BY oi.order_id  
)  
SELECT *  
FROM order_totals  
WHERE total_amount >= 1000;
```

Validating and debugging CTE queries

- Test this first:

```
SELECT
    oi.order_id,
    SUM(oi.quantity * oi.unit_price) AS total_amount
FROM order_items oi
GROUP BY oi.order_id;
```

Query the CTE directly before adding more joins or filters

- Start simple:

```
WITH order_totals AS (  
    SELECT  
        oi.order_id,  
        SUM(oi.quantity * oi.unit_price) AS total_amount  
    FROM order_items oi  
    GROUP BY oi.order_id  
)  
SELECT *  
FROM order_totals;
```


Query the CTE directly before adding more joins or filters

- Then add the filter:

```
WITH order_totals AS (  
    SELECT  
        oi.order_id,  
        SUM(oi.quantity * oi.unit_price) AS total_amount  
    FROM order_items oi  
    GROUP BY oi.order_id  
)  
SELECT *  
FROM order_totals  
WHERE total_amount >= 1000;
```

Query the CTE directly before adding more joins or filters

- Then add joins if needed:

```
-- This helps isolate where the problem starts.
WITH order_totals AS (
    SELECT
        oi.order_id,
        SUM(oi.quantity * oi.unit_price) AS total_amount
    FROM order_items oi
    GROUP BY oi.order_id
)
SELECT
    o.order_id,
    o.customer_id,
    ot.total_amount
FROM orders o
JOIN order_totals ot
    ON ot.order_id = o.order_id
WHERE ot.total_amount >= 1000;
```

Check column names carefully

- Common bug is referencing a column that does not exist in the CTE.

```
WITH order_totals AS (  
    SELECT  
        oi.order_id,  
        SUM(oi.quantity * oi.unit_price) AS total_amount  
    FROM order_items oi  
    GROUP BY oi.order_id  
)  
SELECT customer_id  
FROM order_totals;
```

For recursive CTEs, check anchor and recursive parts separately

*-- does $n + 1$ move toward the stopping condition?
-- does WHERE $n < 10$ actually stop recursion?*

```
WITH RECURSIVE num_series AS (  
    SELECT 1 AS n  
    UNION ALL  
    SELECT n + 1  
    FROM num_series  
)  
SELECT *  
FROM num_series;
```

For recursive CTEs, check anchor and recursive parts separately

```
mysql> WITH RECURSIVE num_series AS (  
->     SELECT 1 AS n  
->     UNION ALL  
->     SELECT n + 1  
->     FROM num_series  
-> )  
-> SELECT *  
-> FROM num_series;  
ERROR 3636 (HY000): Recursive query aborted after 1001 iterations. Try increasing @@cte_max_recursion_depth to a larger value.
```

Advanced Aggregations and Reporting SQL

GROUP BY behavior in MySQL

- **GROUP BY** aggregates rows into groups based on one or more columns.
- You can think: "**Collapse multiple rows into one row per group**"
- **Logical Processing**
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Group By Core Rule

- Every column in SELECT must be:
 - Either in GROUP BY if **functionality dependent**
 - Or wrapped in an aggregate function if **not functionality dependent**

Group By Core Rule

```
-- Error (ONLY_FULL_GROUP_BY enabled)  
-- order_date is not functionally dependent
```

```
SELECT
```

```
    customer_id,  
    order_date
```

```
FROM orders
```

```
GROUP BY customer_id;
```

```
-- Corrected: Get the latest order per customer
```

```
SELECT
```

```
    customer_id,  
    MAX(order_date) AS last_order
```

```
FROM orders
```

```
GROUP BY customer_id;
```

Special Behavior (ONLY_FULL_GROUP_BY)

- MySQL has two modes
 1. **Strict Mode (Recommended)**
 - Enforces SQL standard
 - Prevents ambiguous results
 2. **Non-Strict Mode**
 - MySQL picks random value for non-grouped columns

Special Behavior (ONLY_FULL_GROUP_BY)

Non-Strict Mode

```
SELECT
    customer_id,
    order_date
FROM orders
GROUP BY customer_id;
```

👉 Might return **unpredictable order_date**

⚠️ Dangerous in production

Functional Dependency

- MySQL allows this:

```
SELECT
    customer_id,
    first_name
FROM customers
GROUP BY customer_id;
```

✓ Why valid?

- customer_id is PK
- first_name is functionally dependent
- One customer_id → exactly one first_name

GROUP BY vs DISTINCT

-- *DISTINCT*

```
SELECT DISTINCT customer_id  
FROM orders;
```

-- *GROUP BY*

-- ✓ *Same result*

-- *But GROUP BY allows aggregation*

```
SELECT customer_id  
FROM orders  
GROUP BY customer_id;
```

Group By Performance Behavior

- GROUP BY can trigger
 - Temporary tables
 - Filesort
 - Full scan (if no index)
- Typical execution
 1. Scan rows
 2. Sort or hash group
 3. Aggregate per group
 4. Return result

Aggregation strategies

- Usually this is about questions like:
 - Total sales
 - Number of orders
 - Average product price
 - Highest order value
 - Per customer, per city, per month, per product category

Aggregation strategies

1. **Whole-table aggregation** - This summarizes the entire result set into one row.

Example: total number of orders

```
SELECT COUNT(*) AS total_orders  
FROM orders;
```

Example: total sales amount from all order items

```
SELECT SUM(quantity * unit_price) AS total_sales  
FROM order_items;
```


Aggregation strategies

2. Grouped aggregation - this summarizes rows per group.

Example: total orders per customer

```
SELECT
    customer_id,
    COUNT(*) AS order_count
FROM orders
GROUP BY customer_id;
```

Example: sales per product

```
SELECT
    product_id,
    SUM(quantity * unit_price) AS product_sales
FROM order_items
GROUP BY product_id;
```



Aggregation strategies

3. Multi-level grouping - You can aggregate by more than one column.

Example: orders per city and status

```
SELECT
    c.city,
    o.status,
    COUNT(*) AS total_orders
FROM customers c
JOIN orders o
    ON o.customer_id = c.customer_id
GROUP BY
    c.city,
    o.status;
```

Aggregation strategies

4. Conditional aggregation - This is one of the most useful strategies. You aggregate only rows that meet a condition, usually with CASE.

Example: shipped vs pending orders in one query

```
SELECT
    COUNT(*) AS total_orders,
    SUM(CASE WHEN status = 'Shipped' THEN 1 ELSE 0 END) AS shipped_orders,
    SUM(CASE WHEN status = 'Pending' THEN 1 ELSE 0 END) AS pending_orders,
    SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_orders
FROM orders;
```

Aggregation strategies

4. **Conditional aggregation** - This is one of the most useful strategies. You aggregate only rows that meet a condition, usually with CASE.

Example: total sales only for shipped orders

```
SELECT
    SUM(
        CASE
            WHEN o.status = 'Shipped' THEN oi.quantity * oi.unit_price
            ELSE 0
        END
    ) AS shipped_sales
FROM orders o
JOIN order_items oi
    ON oi.order_id = o.order_id;
```



Aggregation strategies

5. **Aggregation after joins** - Sometimes the aggregation must happen after combining tables.

Example: total sales per customer

```
SELECT c.customer_id, c.first_name, c.last_name,  
       SUM(oi.quantity * oi.unit_price) AS total_sales  
FROM customers c  
JOIN orders o ON o.customer_id = c.customer_id  
JOIN order_items oi ON oi.order_id = o.order_id  
GROUP BY  
    c.customer_id,  
    c.first_name,  
    c.last_name;
```



Aggregation strategies

6. **Pre-aggregation before join** - Sometimes it is better to aggregate first, then join later.

Example: compute order totals first, then join to orders/customers

```
WITH order_totals AS (  
    SELECT order_id, SUM(quantity * unit_price) AS order_total  
    FROM order_items  
    GROUP BY order_id  
)  
SELECT o.order_id, o.customer_id, ot.order_total  
FROM orders o  
JOIN order_totals ot ON ot.order_id = o.order_id;
```



Activity 3 - Stage Aggregation

- Create an aggregation strategies that will execute in stage structure

Flow

Step 1: order_items → order totals

Step 2: order totals → customer totals

Step 3: customer totals → city totals

Activity 3 - Stage Aggregation

customers (6r × 2c)		
#	1 city 	2 city_total_sales
1	(NULL)	1,741,740.09
2	Bacolod	6,974,935.63
3	Cebu	6,842,795.0
4	Davao	10,428,948.67
5	Iloilo	8,682,501.4
6	Manila	3,497,921.19

Activity 3 - Stage Aggregation

```
-- Get the total orders per order items
WITH order_totals AS (
    SELECT
        order_id,
        SUM(quantity * unit_price) AS order_total
    FROM order_items
    GROUP BY order_id
),
```

Activity 3 - Stage Aggregation

```
-- Get the order per city
SELECT
    c.city,
    SUM(ct.customer_total) AS city_total_sales
FROM customer_totals ct
JOIN customers c
    ON c.customer_id = ct.customer_id
GROUP BY c.city;
```

Activity 3 - Stage Aggregation

```
-- Get the order per city
SELECT
    c.city,
    SUM(ct.customer_total) AS city_total_sales
FROM customer_totals ct
JOIN customers c
    ON c.customer_id = ct.customer_id
GROUP BY c.city;
```

ORDER BY and LIMIT usage

- Logical Processing
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
- Sorting happens before limiting
- LIMIT is applied after sorting
- Logical Presentation
 1. Read rows already in sorted order (via index)
 2. Stop as soon as LIMIT is satisfied

ORDER BY and LIMIT usage

- General Syntax

```
SELECT column1, column2  
FROM table_name  
ORDER BY column_name [ASC | DESC]  
LIMIT offset, row_count;
```

ORDER BY and LIMIT usage

- Top Customers by Spending

```
SELECT
    o.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_spent
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.customer_id
ORDER BY total_spent DESC
LIMIT 10;
```

Common Mistakes of ORDER BY and LIMIT

- Random Rows (Avoid on Large Data ⚠)

```
SELECT *  
FROM customers  
ORDER BY RAND()  
LIMIT 5;
```

```
mysql> SELECT *
-> FROM customers
-> ORDER BY RAND()
-> LIMIT 5;
```

customer_id	first_name	last_name	email	city	created_at
531	first531	last531	user531@example.com	Cebu	2024-10-31 10:34:44
607	first607	last607	user607@example.com	Davao	2024-08-16 10:34:44
388	first388	last388	user388@example.com	Iloilo	2025-03-23 10:34:44
239	first239	last239	user239@example.com	Bacolod	2025-08-19 10:34:44
900	first900	last900	NULL	NULL	NULL

5 rows in set (0.01 sec)

```
mysql> SELECT *
-> FROM customers
-> ORDER BY RAND()
-> LIMIT 5;
```

customer_id	first_name	last_name	email	city	created_at
473	first473	last473	user473@example.com	Iloilo	2024-12-28 10:34:44
70	first70	last70	user70@example.com	Manila	2026-02-04 10:34:44
201	first201	last201	user201@example.com	Cebu	2025-09-26 10:34:44
311	first311	last311	user311@example.com	Cebu	2025-06-08 10:34:44
698	first698	last698	user698@example.com	Iloilo	2024-05-17 10:34:44

5 rows in set (0.00 sec)

Common Mistakes of ORDER BY and LIMIT

-- Missing ORDER BY

```
SELECT * FROM orders LIMIT 10;
```

-- Using LIMIT before ORDER BY

```
SELECT * FROM orders LIMIT 10 ORDER BY order_id;
```

-- Large OFFSET (Pagination Problem)

-- MySQL still scans first 100k rows → slow

```
SELECT * FROM orders LIMIT 10000 ORDER BY order_id;
```

Top-N per group

- Return the **top N rows inside each group**, *not just top N rows for the whole table*.
- You want:
 - Top 3 most expensive products per category
 - Latest 2 orders per customer
 - Top 1 best-selling product per category

Top-N per group logical pattern

```
SELECT *
FROM (
    SELECT
        ...,
        ROW_NUMBER() OVER (
            PARTITION BY group_column
            ORDER BY sort_column DESC
        ) AS rn
    FROM table_name
) AS x
WHERE x.rn <= N;
```

Top 3 most expensive products per category

```
-- Version: Derived Table + Window Functions (20% faster to CTE)
SELECT
    ranked.category,
    ranked.product_id,
    ranked.product_name,
    ranked.price,
    ranked.row_num
FROM (
    SELECT
        p.category,
        p.product_id,
        p.product_name,
        p.price,
        ROW_NUMBER() OVER (
            PARTITION BY p.category
            ORDER BY p.price DESC
        ) AS row_num
    FROM products p
) AS ranked
WHERE ranked.row_num <= 3
ORDER BY ranked.category, ranked.row_num;
```



Top 3 most expensive products per category

```
-- Version: CTE + Window Functions (More readable)
WITH ranked AS (
    SELECT
        p.category,
        p.product_id,
        p.product_name,
        p.price,
        ROW_NUMBER() OVER (
            PARTITION BY p.category
            ORDER BY p.price DESC
        ) AS row_num
    FROM products p
)
SELECT
    ranked.category,
    ranked.product_id,
    ranked.product_name,
    ranked.price,
    ranked.row_num
FROM ranked
WHERE ranked.row_num <= 3
ORDER BY ranked.category, ranked.row_num;
```

Top 3 most expensive products per category

```
-- Version: Correlated Subquery (50x-75x slower to CTE)
SELECT
    p1.category,
    p1.product_id,
    p1.product_name,
    p1.price
FROM products p1
WHERE (
    SELECT COUNT(*)
    FROM products p2
    WHERE p2.category = p1.category
        AND p2.price > p1.price
) < 3
ORDER BY p1.category, p1.price DESC;
```

Activity 4 - Latest 2 Orders per Customer

- Create a report that will show the top latest orders of customer
- Sort by order_date



orders (1,952r × 5c)

#	1 customer_id	2 order_id	3 order_date	4 status	5 row_num
1	(NULL)	8,760	2026-04-15 10:34:44	Pending	1
2	(NULL)	5,840	2026-04-15 10:34:44	Pending	2
3	2	1	2026-04-14 10:34:44	Shipped	1
4	2	7,001	2026-02-08 10:34:44	Shipped	2
5	3	2	2026-04-13 10:34:44	Delivered	1
6	3	7,002	2026-02-07 10:34:44	Delivered	2
7	4	3	2026-04-12 10:34:44	Cancelled	1
8	4	7,003	2026-02-06 10:34:44	Cancelled	2
9	5	4	2026-04-11 10:34:44	Pending	1
10	5	7,004	2026-02-05 10:34:44	Pending	2
11	6	5	2026-04-10 10:34:44	Shipped	1
12	6	7,005	2026-02-04 10:34:44	Shipped	2
13	7	6	2026-04-09 10:34:44	Delivered	1



SELECT

ranked.customer_id,
ranked.order_id,
ranked.order_date,
ranked.status,
ranked.row_num

FROM (

SELECT

o.customer_id,
o.order_id,
o.order_date,
o.status,
ROW_NUMBER() **OVER** (
 PARTITION BY o.customer_id
 ORDER BY o.order_date **DESC**, o.order_id **DESC**
) AS row_num

FROM orders o

) AS ranked

WHERE ranked.row_num <= 2

ORDER BY ranked.customer_id, ranked.row_num;



DISTINCT counting correctly

- **DISTINCT counting correctly** means understanding what exactly is being de-duplicated before counting.
- For example

```
-- Count non-NULL and unique products that appear in order_items  
SELECT COUNT(DISTINCT product_id) AS ordered_products  
FROM order_items;
```

```
-- Count how many cities have customers  
SELECT COUNT(DISTINCT city) AS unique_cities  
FROM customers;
```

DISTINCT counting correctly

```
-- Per-order distinct counting
SELECT
    order_id,
    COUNT(product_id) AS total_product_rows,
    COUNT(DISTINCT product_id) AS unique_products
FROM order_items
GROUP BY order_id;
```

DISTINCT counting correctly

```
-- Multi-column DISTINCT count in  
-- Select the order_items that have unique combination of order_id and  
product_id
```

```
SELECT  
COUNT(DISTINCT order_id, product_id) AS unique_order_product_pairs  
FROM order_items;
```

DISTINCT counting correctly

- **Wrong Distinct Pattern**

*-- Instead of counting the distinct city, it count the city
-- then return the number therefore the distinct function return the same number.*

```
SELECT DISTINCT COUNT(city) AS city_count  
FROM customers;
```

Avoiding double-counting

- Double-counting happens when **one row in a parent table joins to multiple rows in a child table.**
- Example:
 - 1 order → 5 order_items
 - ↳ JOIN = 5 rows
 - ↳ SUM(order_amount) gets multiplied ×5 **✗**

Example of double-counting

```
-- COUNT(o.order_id) becomes X3, instead of actual order count is only 1
SELECT
    c.customer_id,
    COUNT(o.order_id) AS order_count
FROM customers c
JOIN orders o
    ON o.customer_id = c.customer_id
JOIN order_items oi
    ON oi.order_id = o.order_id -- Inflated
GROUP BY c.customer_id;
```

Example of double-counting

```
-- To fix
-- Option 1: Just remove any joints that is not needed for reporting
SELECT
    c.customer_id,
    COUNT(o.order_id) AS order_count
FROM customers c
JOIN orders o
    ON o.customer_id = c.customer_id
GROUP BY c.customer_id;
```


Example of double-counting

-- Option 2: Using the COUNT(DISTINCT()) to reduce the output but this will cause heavy resource consumption

```
SELECT
    c.customer_id,
    COUNT(DISTINCT o.order_id) AS order_count
FROM customers c
JOIN orders o
    ON o.customer_id = c.customer_id
JOIN order_items oi
    ON oi.order_id = o.order_id
GROUP BY c.customer_id;
```

Example of double-counting

```
-- Option 3: Using WITH + Aggregation
WITH customer_orders AS (
    SELECT
        o.order_id,
        o.customer_id
    FROM orders o
)
SELECT
    customer_id,
    COUNT(*) AS order_count
FROM customer_orders
GROUP BY customer_id;
```

Date-based reporting

- `DATE()` → gets only the date part from a `DATETIME`
- `YEAR()` → gets the year number
- `MONTH()` → gets the month number
- `DATE_FORMAT()` → formats a date for reporting output

Example of Date-based Reporting

```
-- DATE() for daily reporting  
-- Use this when order_date contains time, but your report should  
group by calendar date.  
-- i.e. order_date is 2026-01-01 08:15:00 and 2026-01-01 14:40:00,  
both become 2026-01-01.
```

```
SELECT  
    DATE(order_date) AS order_day,  
    COUNT(*) AS total_orders  
FROM orders  
GROUP BY DATE(order_date)  
ORDER BY order_day;
```

Example of Date-based Reporting

```
-- YEAR() for yearly reporting  
-- How many orders were placed per year?
```

```
SELECT  
    YEAR(order_date) AS order_year,  
    COUNT(*) AS total_orders  
FROM orders  
GROUP BY YEAR(order_date)  
ORDER BY order_year;
```

Example of Date-based Reporting

```
-- MONTH() for monthly reporting
-- This gives the month number only.

SELECT
    YEAR(order_date) AS order_year,
    MONTH(order_date) AS order_month,
    COUNT(*) AS total_orders
FROM orders
GROUP BY
    YEAR(order_date),
    MONTH(order_date)
ORDER BY
    order_year,
    order_month;
```

Example of Date-based Reporting

```
-- DATE_FORMAT() for report-friendly labels  
-- Monthly label
```

```
SELECT  
    DATE_FORMAT(order_date, '%Y-%m') AS report_month,  
    COUNT(*) AS total_orders  
FROM orders  
GROUP BY DATE_FORMAT(order_date, '%Y-%m')  
ORDER BY report_month;
```

```
-- More readable month label
```

```
SELECT  
    DATE_FORMAT(order_date, '%b %Y') AS report_month,  
    COUNT(*) AS total_orders  
FROM orders  
GROUP BY DATE_FORMAT(order_date, '%b %Y')  
ORDER BY MIN(order_date);
```



Window Functions

Window function fundamentals

- A window function performs calculations **across a set of rows related to the current row**, without collapsing rows like GROUP BY.

Feature	GROUP BY	Window Function
Output rows	Reduced	Preserved
Aggregation	Yes	Yes
Row detail	Lost	Retained

Window function fundamentals

- Basic Syntax

```
function_name(...)  
OVER (  
    PARTITION BY column  
    ORDER BY column  
)
```

- ✓ **PARTITION BY** - Like GROUP BY, but does NOT collapse rows
- ✓ **ORDER BY** - Defines the order inside each partition
- ✓ **Window Frame (Optional)** - Defines range of rows for calculation

Example of Window function

```
-- Total order amount per order
-- Partition = per customer
-- Ordered by DATE
-- Running total computed per row
SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    SUM(oi.quantity * oi.unit_price)
        OVER (PARTITION BY o.customer_id ORDER BY o.order_date) AS running_total
FROM orders o
JOIN order_items oi
    ON oi.order_id = o.order_id;
```

Ranking Functions

Feature	ROW_NUMBER()	RANK()	DENSE_RANK()
Duplicates (ties)	Always unique	Same rank for ties	Same rank for ties
Gap in ranking?	✗ No gaps	☑ Yes (skips numbers)	✗ No gaps
Sequence behavior	Strict sequence (1,2,3,4...)	Skips after ties (1,2,3,3,5)	Continuous (1,2,3,3,4)
Use case	Unique row numbering	Competition ranking	Group ranking
Deterministic?	✗ Not if ties (unless extra ORDER BY)	☑ Yes	☑ Yes

Ranking Functions

product_name	price	ROW_NUMBER()	RANK()	DENSE_RANK()
E	400	1	1	1
D	300	2	2	2
B	200	3	3	3
C	200	4	3	3
A	100	5	5	4

Example - ROW_NUMBER()

```
-- ROW_NUMBER()
-- Top-N per group
WITH Top3Customers AS (
SELECT
    o.customer_id,
    o.order_id,
    o.order_date,
    ROW_NUMBER() OVER (
        PARTITION BY o.customer_id
        ORDER BY o.order_date DESC
    ) AS rank_number
FROM orders o
)
SELECT * FROM Top3Customers WHERE rank_number <= 3;
```

Example - ROW_NUMBER()

	1 customer_id	2 order_id	3 order_date	4 rank_number
4	2	1	2026-04-14 10:34:44	1
5	2	7,001	2026-02-08 10:34:44	2
6	2	3,001	2026-01-24 10:34:44	3
7	3	2	2026-04-13 10:34:44	1
8	3	7,002	2026-02-07 10:34:44	2
9	3	3,002	2026-01-23 10:34:44	3
10	4	3	2026-04-12 10:34:44	1
11	4	7,003	2026-02-06 10:34:44	2
12	4	3,003	2026-01-22 10:34:44	3
13	5	4	2026-04-11 10:34:44	1
14	5	7,004	2026-02-05 10:34:44	2
15	5	3,004	2026-01-21 10:34:44	3
16	6	5	2026-04-10 10:34:44	1

Example - RANK() vs DENSE_RANK()

```
-- RANK() vs DENSE_RANK()
SELECT
    o.customer_id,
    o.order_id,
    RANK() OVER (
        PARTITION BY o.customer_id
        ORDER BY o.order_date DESC
    ) AS rank_val,
    DENSE_RANK() OVER (
        PARTITION BY o.customer_id
        ORDER BY o.order_date DESC
    ) AS dense_rank_val
FROM orders o;
```


Example - RANK() vs DENSE_RANK()

orders (10,000r x 4c)				
#	1 customer_id	2 order_id	3 rank_val	4 dense_rank_val
1	(NULL)	5,840	1	1
2	(NULL)	8,760	1	1
3	(NULL)	2,920	1	1
4	(NULL)	2,560	4	2
5	(NULL)	5,480	4	2
6	(NULL)	8,040	6	3
7	(NULL)	2,200	6	3
8	(NULL)	5,120	6	3
9	(NULL)	1,840	9	4
10	(NULL)	7,680	9	4
11	(NULL)	7,320	11	5
12	(NULL)	1,480	11	5
13	(NULL)	4,400	11	5



Comparison: LAG() vs LEAD()

Feature	LAG()	LEAD()
Direction	Looks backward (previous row)	Looks forward (next row)
Use case	Compare with previous value	Compare with next value
Common scenario	Growth vs previous record	Predict / compare upcoming record
Offset default	1 row before	1 row after
NULL behavior	NULL if no previous row	NULL if no next row

Example - LEAD() and LAG

```
-- Retrieve the previous order and next order date
SELECT
    o.customer_id,
    o.order_date,
    LAG(o.order_date) OVER (
        PARTITION BY o.customer_id
        ORDER BY o.order_date
    ) AS previous_order_date,
    LEAD(o.order_date) OVER (
        PARTITION BY o.customer_id
        ORDER BY o.order_date
    ) AS next_order_date
FROM orders o
WHERE
    o.order_date IS NOT NULL
AND o.customer_id IS NOT NULL;
```



Activity 5 - Sales Changes and Growth Report

- Create a sales growth report

	1 customer_id	2 order_id	3 order_date	4 order_total	5 prev_order_total	6 next_order_total	7 change_from_previous	8 percent_previous
1	2	4,001	2025-04-29 10:34:44	143.66	(NULL)	143.66	(NULL)	(NULL)
2	2	8,001	2025-05-14 10:34:44	143.66	143.66	143.66	0.0	0.0
3	2	1,001	2025-07-18 10:34:44	143.66	143.66	143.66	0.0	0.0
4	2	5,001	2025-08-02 10:34:44	143.66	143.66	143.66	0.0	0.0
5	2	9,001	2025-08-17 10:34:44	143.66	143.66	143.66	0.0	0.0
5	2	2,001	2025-10-21 10:34:44	143.66	143.66	143.66	0.0	0.0
7	2	6,001	2025-11-05 10:34:44	143.66	143.66	96.38	0.0	0.0
8	2	3,001	2026-01-24 10:34:44	96.38	143.66	143.66	-47.28	-32.91
9	2	7,001	2026-02-08 10:34:44	143.66	96.38	143.66	47.28	49.06
0	2	1	2026-04-14 10:34:44	143.66	143.66	(NULL)	0.0	0.0
1	3	4,002	2025-04-28 10:34:44	130.94	(NULL)	130.94	(NULL)	(NULL)
2	3	8,002	2025-05-13 10:34:44	130.94	130.94	130.94	0.0	0.0
3	3	1,002	2025-07-17 10:34:44	130.94	130.94	130.94	0.0	0.0

```
WITH order_totals AS (  
    SELECT  
        o.order_id,  
        o.customer_id,  
        o.order_date,  
        SUM(oi.quantity * oi.unit_price) AS order_total  
    FROM orders o  
    JOIN order_items oi  
        ON oi.order_id = o.order_id  
    WHERE  
        o.customer_id IS NOT NULL AND  
        o.order_date IS NOT NULL  
    GROUP BY  
        o.order_id,  
        o.customer_id,  
        o.order_date  
)
```

SELECT

customer_id,
order_id,
order_date,
order_total,

-- Previous order total

LAG(order_total) **OVER** (
 PARTITION BY customer_id
 ORDER BY order_date
) **AS** prev_order_total,

-- Next order total

LEAD(order_total) **OVER** (
 PARTITION BY customer_id
 ORDER BY order_date
) **AS** next_order_total,

```

-- Change vs previous
order_total - LAG(order_total) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
) AS change_from_previous,

-- % growth vs previous
ROUND (
    (
        order_total - LAG(order_total) OVER (
            PARTITION BY customer_id
            ORDER BY order_date
        )
    )
    / NULLIF(LAG(order_total) OVER (-- Null if 0
        PARTITION BY customer_id
        ORDER BY order_date
    ), 0) * 100,
    2
) AS percent_previous

```

```

FROM order_totals
ORDER BY customer_id, order_date;

```

Comparison: FIRST_VALUE() vs LAST_VALUE()

- **FIRST_VALUE()**

- Returns the first value in the window (based on ORDER BY).
- Always returns the earliest (or first) value in the partition
- Works as expected without extra configuration

- **LAST_VALUE()**

- Return the last value in the window
- Window includes all rows
- Returns the true last value in the partition

Example - FIRST_VALUE() vs LAST_VALUE()

```
-- Return the first and last order of the customer
SELECT
    o.customer_id,
    o.order_date,
    FIRST_VALUE(o.order_date) OVER (
        PARTITION BY o.customer_id
        ORDER BY o.order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS first_order,
    LAST_VALUE(o.order_date) OVER (
        PARTITION BY o.customer_id
        ORDER BY o.order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_order
FROM orders o;
```

Example - FIRST_VALUE() vs LAST_VALUE()

orders (10,000r × 4c)				
	1 customer_id	2 order_date	3 first_order	4 last_order
9988	999	2025-11-23 10:34:44	2025-05-02 10:34:44	2026-02-11 10:34:44
9989	999	2026-01-27 10:34:44	2025-05-02 10:34:44	2026-02-11 10:34:44
9990	999	2026-02-11 10:34:44	2025-05-02 10:34:44	2026-02-11 10:34:44
9991	1,000	2025-05-01 10:34:44	2025-05-01 10:34:44	2026-02-10 10:34:44
9992	1,000	2025-05-16 10:34:44	2025-05-01 10:34:44	2026-02-10 10:34:44
9993	1,000	2025-07-20 10:34:44	2025-05-01 10:34:44	2026-02-10 10:34:44
9994	1,000	2025-08-04 10:34:44	2025-05-01 10:34:44	2026-02-10 10:34:44
9995	1,000	2025-08-19 10:34:44	2025-05-01 10:34:44	2026-02-10 10:34:44
9996	1,000	2025-10-23 10:34:44	2025-05-01 10:34:44	2026-02-10 10:34:44
9997	1,000	2025-11-07 10:34:44	2025-05-01 10:34:44	2026-02-10 10:34:44
9998	1,000	2025-11-22 10:34:44	2025-05-01 10:34:44	2026-02-10 10:34:44
9999	1,000	2026-01-26 10:34:44	2025-05-01 10:34:44	2026-02-10 10:34:44
10000	1,000	2026-02-10 10:34:44	2025-05-01 10:34:44	2026-02-10 10:34:44

Running totals and cumulative metrics

- **Running totals and cumulative metrics** are window-function patterns where each row includes a result based on the current row plus earlier rows in some defined order.

Example - Running total

```
-- A running total keeps adding values row by row.
WITH order_totals AS (
    SELECT
        o.order_id,
        o.order_date,
        SUM(oi.quantity * oi.unit_price) AS order_total
    FROM orders o
    JOIN order_items oi
        ON oi.order_id = o.order_id
    GROUP BY
        o.order_id,
        o.order_date
)
SELECT
    order_id,
    order_date,
    order_total,
    SUM(order_total) OVER (
        ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM order_totals
ORDER BY order_date;
```

Example - Running total

orders (10,000r × 4c)				
#	1 order_id	2 order_date	3 order_total	4 running_total
1	35	(NULL)	727.0	727.0
2	70	(NULL)	1,364.0	2,091.0
3	105	(NULL)	814.4	2,905.4
4	140	(NULL)	2,638.0	5,543.4
5	175	(NULL)	3,275.0	8,818.4
6	210	(NULL)	3,912.0	12,730.4
7	245	(NULL)	4,549.0	17,279.4
8	280	(NULL)	5,186.0	22,465.4
9	315	(NULL)	5,823.0	28,288.4
10	350	(NULL)	2,163.0	30,451.4
11	385	(NULL)	3,457.0	33,908.4
12	420	(NULL)	4,094.0	38,002.4
13	455	(NULL)	4,731.0	42,733.4

Example - Cumulative Percentage

```
-- Cumulative percentage of orders
WITH daily_sales_cte AS (
    SELECT
        o.order_date,
        SUM(oi.quantity * oi.unit_price) AS daily_sales
    FROM orders o
    JOIN order_items oi
        ON oi.order_id = o.order_id
    GROUP BY o.order_date
)
SELECT
    order_date,
    daily_sales,

    -- Running total
    SUM(daily_sales) OVER (
        ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_sales,
```



Example - Cumulative Percentage

```
-- Grand total
SUM(daily_sales) OVER () AS total_sales,

-- Cumulative percentage
ROUND (
    (
        SUM(daily_sales) OVER (
            ORDER BY order_date
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        )
        -- Compute the SUM over the entire result set (no partition, no
order, no frame).
        / NULLIF(SUM(daily_sales) OVER (), 0)
    ) * 100,
    2
) AS cumulative_percentage
FROM daily_sales_cte
ORDER BY order_date;
```

Period comparisons

- In SQL, this is commonly used for
 - Today vs yesterday
 - This month vs last month
 - This order vs previous order
 - This year vs last year

Example - Period Comparison

```
-- Compare monthly sales with the previous month
WITH monthly_sales AS (
    SELECT
        YEAR(o.order_date) AS sales_year,
        MONTH(o.order_date) AS sales_month,
        SUM(oi.quantity * oi.unit_price) AS monthly_total
    FROM orders o
    JOIN order_items oi
        ON oi.order_id = o.order_id
    GROUP BY
        YEAR(o.order_date),
        MONTH(o.order_date)
)
```

Example - Period Comparison

```
SELECT
    sales_year,
    sales_month,
    monthly_total,
    LAG(monthly_total) OVER (
        ORDER BY sales_year, sales_month
    ) AS previous_month_total,
    monthly_total - LAG(monthly_total) OVER (
        ORDER BY sales_year, sales_month
    ) AS month_difference
FROM monthly_sales
ORDER BY sales_year, sales_month;
```

Activity 6 - Period Comparisons

- Create a monthly sales as percentage change from previous month

previous (14r × 6c)						
#	1 sales_year	2 sales_month	3 monthly_total	4 previous_month_total	5 amount_change	6 percent_change
1	(NULL)	(NULL)	1,115,626.9	(NULL)	(NULL)	(NULL)
2	2,025	4	1,461,019.42	1,115,626.9	345,392.52	30.96
3	2,025	5	3,098,889.72	1,461,019.42	1,637,870.3	112.1
4	2,025	6	3,026,381.53	3,098,889.72	-72,508.19	-2.34
5	2,025	7	3,096,576.12	3,026,381.53	70,194.59	2.32
6	2,025	8	3,262,106.4	3,096,576.12	165,530.28	5.35
7	2,025	9	3,138,935.96	3,262,106.4	-123,170.44	-3.78
8	2,025	10	3,247,169.24	3,138,935.96	108,233.28	3.45
9	2,025	11	3,245,538.98	3,247,169.24	-1,630.26	-0.05
10	2,025	12	3,383,475.97	3,245,538.98	137,936.99	4.25
11	2,026	1	3,284,525.6	3,383,475.97	-98,950.37	-2.92
12	2,026	2	3,016,292.0	3,284,525.6	-268,233.6	-8.17
13	2,026	3	3,189,996.16	3,016,292.0	173,704.16	5.76



Activity 6 - Period Comparisons

```
-- Compare monthly sales as percentage change from previous month
WITH monthly_sales AS (
    SELECT
        YEAR(o.order_date) AS sales_year,
        MONTH(o.order_date) AS sales_month,
        SUM(oi.quantity * oi.unit_price) AS monthly_total
    FROM orders o
    JOIN order_items oi
        ON oi.order_id = o.order_id
    GROUP BY
        YEAR(o.order_date),
        MONTH(o.order_date)
)
```

Activity 6 - Period Comparisons

```
SELECT
    sales_year,
    sales_month,
    monthly_total,
    LAG(monthly_total) OVER (
        ORDER BY sales_year, sales_month
    ) AS previous_month_total,
    monthly_total - LAG(monthly_total) OVER (
        ORDER BY sales_year, sales_month
    ) AS amount_change,
```

Activity 6 - Period Comparisons

```
ROUND (
    (
        monthly_total - LAG(monthly_total) OVER (
            ORDER BY sales_year, sales_month
        )
    )
    / NULLIF(
        LAG(monthly_total) OVER (
            ORDER BY sales_year, sales_month
        ),
        0
    ) * 100,
    2
) AS percent_change
FROM monthly_sales
ORDER BY sales_year, sales_month;
```



Moving Averages

- It shows the average of a value across a sliding window of rows.
- It is useful for smoothing daily, weekly, or monthly fluctuations.
- For example
 - Current day only → normal value
 - Current day + 2 previous days → 3-row moving average
 - Current month + previous 2 months → 3-period moving average

Moving Averages

- General syntax

```
AVG(expression) OVER (  
    ORDER BY some_column  
    ROWS BETWEEN n PRECEDING AND CURRENT ROW  
)
```

- **AVG(...) OVER (...)** = window average
- **ORDER BY** = defines the sequence
- **ROWS BETWEEN n PRECEDING AND CURRENT ROW** = defines the moving window

Example - Moving Averages

```
-- Moving averages

-- 3-day moving average of daily basis
-- Since its a 3-day window, let say
-- Jan 1 → avg(100) = 100
-- Jan 2 → avg(100, 200) = 150
-- Jan 3 → avg(100, 200, 300) = 200
-- Jan 4 → avg(200, 300, 400) = 300

WITH daily_sales AS (
    SELECT
        DATE(o.order_date) AS sales_date,
        SUM(oi.quantity * oi.unit_price) AS daily_total
    FROM orders o
    JOIN order_items oi
        ON oi.order_id = o.order_id
    GROUP BY DATE(o.order_date)
)
```



Example - Moving Averages

```
SELECT
    sales_date,
    daily_total,
    ROUND (
        AVG(daily_total) OVER (
            ORDER BY sales_date
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        ),
        2
    ) AS moving_avg_3_days
FROM daily_sales
ORDER BY sales_date;
```

Example - Moving Averages

	1 sales_date	2 daily_total	3 moving_avg_3_days
1	(NULL)	1,115,626.9	1,115,626.9
2	2025-04-16	70,654.5	593,140.7
3	2025-04-17	90,080.71	425,454.04
4	2025-04-18	116,251.81	92,329.01
5	2025-04-19	133,780.65	113,371.06
6	2025-04-20	81,915.4	110,649.29
7	2025-04-21	64,704.05	93,466.7
8	2025-04-22	90,169.9	78,929.78
9	2025-04-23	114,379.95	89,751.3
10	2025-04-24	134,099.15	112,883.0
11	2025-04-25	80,887.1	109,788.73
12	2025-04-26	66,078.15	93,688.13
13	2025-04-27	89,264.4	78,743.22

High Availability and Data Recovery

High availability

- High availability (HA) means a system is designed to stay accessible and operational even when something goes wrong, such as:
 - hardware failure
 - software crash
 - network interruption
 - server outage
 - storage failure

Fault tolerance

- **Fault tolerance** means a system can **continue operating immediately** even when a component fails, with little or no interruption.
- Example
 - If one database node crashes but another node instantly continues processing requests, the setup shows fault tolerance.



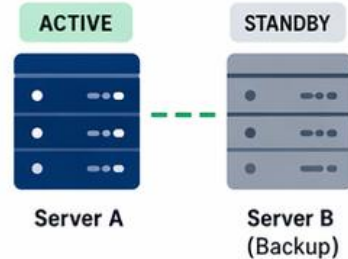
HOW IT WORKS

A fault-tolerant system has **redundant components** ready to take over when a failure occurs.

1

NORMAL OPERATION

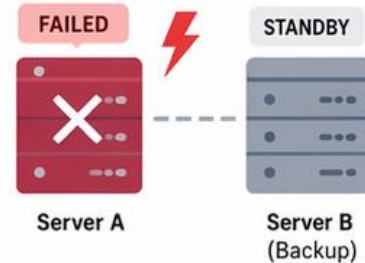
Both components are running. One is active, the other is ready.



2

FAILURE HAPPENS

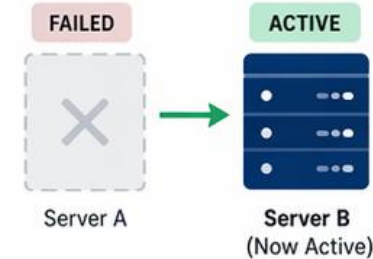
The active component fails.



3

SYSTEM KEEPS RUNNING

The backup automatically takes over. Service continues with little or no interruption.



KEY BENEFITS



MINIMIZES DOWNTIME

Keeps services available even when failures occur.



IMPROVES RELIABILITY

Users can trust the system to stay up and running.



AUTOMATIC RECOVERY

Failover happens quickly without manual intervention.



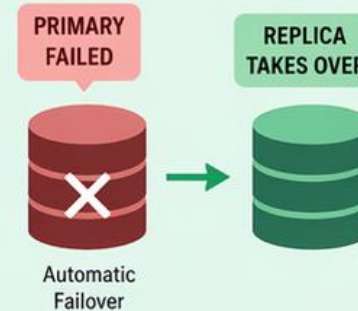
BETTER USER EXPERIENCE

Little to no interruption for users.



REAL-WORLD EXAMPLE

Databases use fault tolerance with replication. If the primary database server fails, a replica can instantly become the primary, and applications continue to work.



Redundancy

- **Redundancy** means having **extra components, resources, or copies** available so the system has a backup when something fails.
- For Example
 - Instead of having only one database server, you have:
 1. Primary database
 2. Secondary database replica



WHAT IS IT?

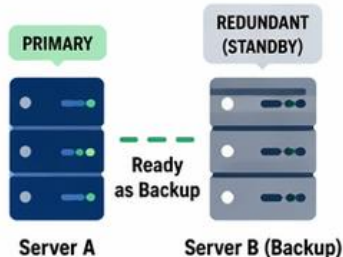
Redundancy is the practice of adding duplicate components so there is a when a failure occurs.



HOW IT WORKS

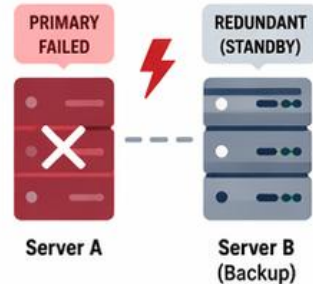
1 NORMAL OPERATION

Both primary and redundant components are available.



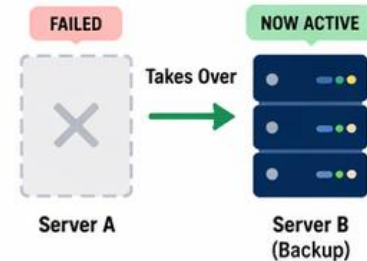
2 FAILURE OCCURS

The primary component fails.



3 BACKUP IS USED

The redundant component is used to keep the system running.



KEY BENEFITS



INCREASES AVAILABILITY

Provides backup so services can continue.



REDUCES RISK

Protects against hardware, network, and component failures.



SUPPORTS QUICK RECOVERY

Speeds up failover and reduces downtime.



IMPROVES RELIABILITY

Builds trust and stability into the system.



REAL-WORLD EXAMPLES



Multiple Servers

If one server fails, another can take over.



Database Replication

A replica provides a copy of the data.



RAID Storage

Data is spread across multiple disks, so it survives a disk failure.



Dual Power Supplies

One power supply can fail while the other keeps the system running.

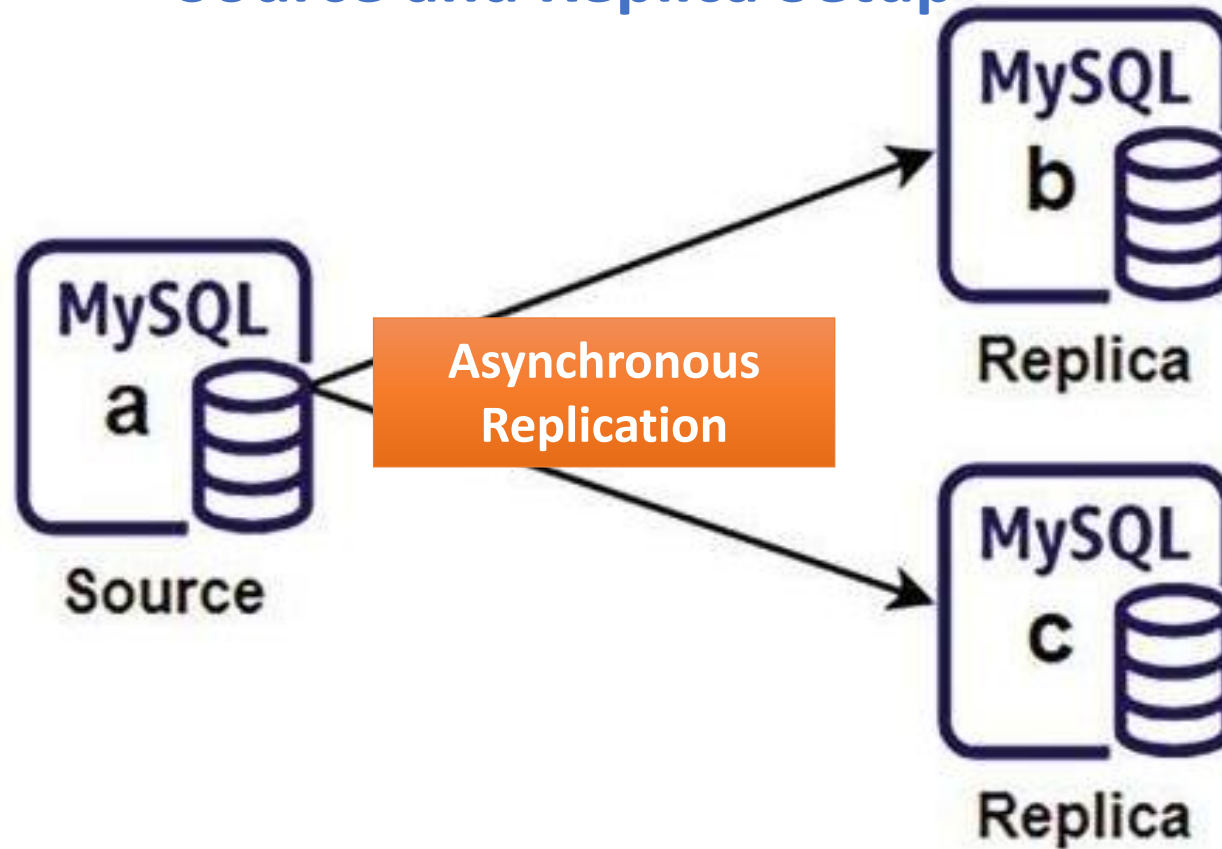


Source-Replica Replication

- **Source** - Formerly the "**Master**" is the primary server that handles write operations and records them in binary logs.
- **Replica** - Formerly the "**Slave**" are the servers that copy and apply the data changes from the Source.
- **Multi-Source Replication** - Formerly "**Multi-Master**" or "**Multi-Slave**" contexts when one replica connects to multiple sources.
- **Multithreaded Applier (MTA)** - Formerly as "**Multithreaded Slave**" (MTS) a threads on the replica that process transactions in parallel.
- **Source Configuration** - Formerly as "Master Configuration," you now configure **Replication Sources**.

MySQL replication using Docker

Source and Replica Setup

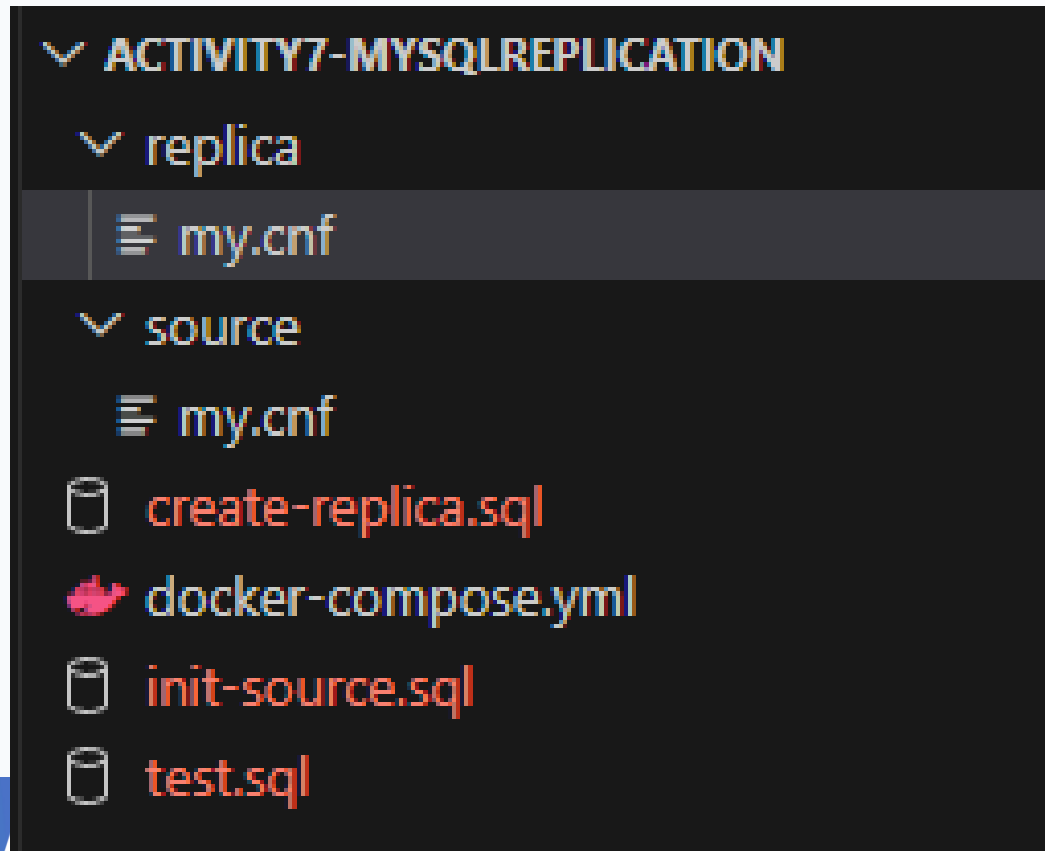


Replication Commands

Old Command (Removed)	New Command (Required in MySQL 8.4)
CHANGE MASTER TO	CHANGE REPLICATION SOURCE TO
START SLAVE	START REPLICA
STOP SLAVE	STOP REPLICA
SHOW SLAVE STATUS	SHOW REPLICA STATUS
SHOW MASTER STATUS	SHOW BINARY LOG STATUS
RESET SLAVE	RESET REPLICA

Activity 7 - MySQL Replication using Docker

1. Prepare the file structure as follows



Activity 7 - MySQL Replication using Docker

2. Update the **File: ./source/my.cnf**

```
# File: ./source/my.cnf

[mysqld]
server-id=1

log_bin=mysql-bin
binlog_format=ROW
gtid_mode=ON
enforce_gtid_consistency=ON

# optional but recommended
log_replica_updates=ON
bind-address=0.0.0.0
```



Activity 7 - MySQL Replication using Docker

3. Update the **File: ./replica/my.cnf**

```
# File: ./replica/my.cnf

[mysqld]
server-id=2

log_bin=mysql-bin
relay_log=relay-bin

gtid_mode=ON
enforce_gtid_consistency=ON
read_only=ON
super_read_only=ON

log_replica_updates=ON
bind-address=0.0.0.0
```



Activity 7 - MySQL Replication using Docker

4. Update the File: ./init-source.sql

```
-- File: ./init-source.sql

CREATE USER 'repl_user'@'%' IDENTIFIED BY 'Admin12345';
GRANT REPLICATION SLAVE, REPLICATION CLIENT ON *.* TO 'repl_user'@'%';
FLUSH PRIVILEGES;
```


Activity 7 - MySQL Replication using Docker

4. Update the File: ./init-source.sql

```
-- File: ./init-source.sql

CREATE USER 'repl_user'@'%' IDENTIFIED BY 'Admin12345';
GRANT REPLICATION SLAVE, REPLICATION CLIENT ON *.* TO 'repl_user'@'%';
FLUSH PRIVILEGES;
```

Activity 7 - MySQL Replication using Docker

5. Update the File: `./docker-compose.yml`

```
docker-compose.yml
1  # Docker composer: docker-composer.yml
   ▶ Run All Services
2  services:
   ▶ Run Service
3    mysql-source:
4      image: mysql:8.4
5      container_name: mysql-source
6      restart: unless-stopped
7      environment:
8        MYSQL_ROOT_PASSWORD: Admin12345
9      ports:
10       - "3307:3306"
11     volumes:
12       - source_data:/var/lib/mysql
13       - ./source/my.cnf:/etc/my.cnf:ro
14       - ./init-source.sql:/docker-entrypoint-initdb.d/init-source.sql
15     networks:
16       - mysql-repl-net
17
```

```
17
18   ▷Run Service
19   mysql-replica:
20     image: mysql:8.4
21     container_name: mysql-replica
22     restart: unless-stopped
23     ports:
24       - "3308:3306"
25     volumes:
26       - replica_data:/var/lib/mysql
27       - ./replica/my.cnf:/etc/my.cnf:ro
28     depends_on:
29       - mysql-source
30     networks:
31       - mysql-repl-net
32
33   volumes:
34     source_data:
35     replica_data:
36
37   networks:
38     mysql-repl-net:
```



Activity 7 - MySQL Replication using Docker

6. Open a terminal and execute command

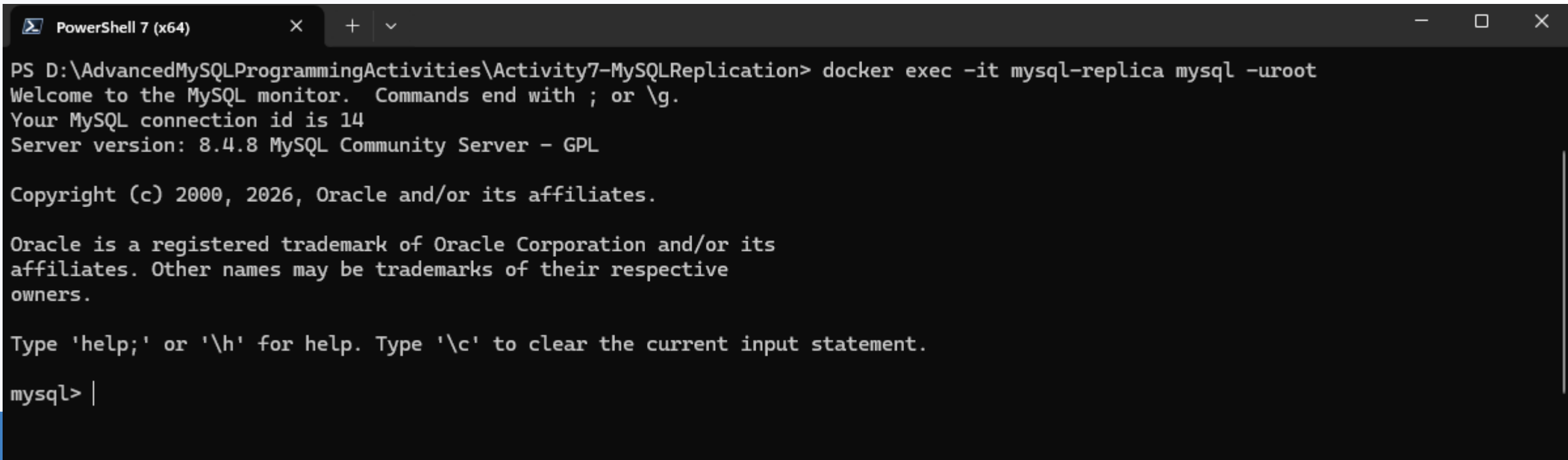
docker compose up -d

```
PowerShell 7 (x64)
PS D:\AdvancedMySQLProgrammingActivities\Activity7-MySQLReplication> docker compose up -d
[+] Running 3/3
✔Network activity7-mysqlreplication_mysql-repl-net  Creat... 0.1s
✔Container mysql-source                            Started 1.0s
✔Container mysql-replica                           Started 1.2s
PS D:\AdvancedMySQLProgrammingActivities\Activity7-MySQLReplication> |
```

Activity 7 - MySQL Replication using Docker

7. In the terminal and execute command

docker exec -it mysql-replica mysql -uroot



```
PowerShell 7 (x64)
PS D:\AdvancedMySQLProgrammingActivities\Activity7-MySQLReplication> docker exec -it mysql-replica mysql -uroot
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 14
Server version: 8.4.8 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> |
```



Activity 7 - MySQL Replication using Docker

8. Inside the **container** of **mysql-replica**, initialize the replica

```
STOP REPLICA;
```

```
CHANGE REPLICATION SOURCE TO
```

```
    SOURCE_HOST = 'mysql-source',
```

```
    SOURCE_PORT = 3306,
```

```
    SOURCE_USER = 'repl_user',
```

```
    SOURCE_PASSWORD = 'Admin12345',
```

```
    SOURCE_AUTO_POSITION = 1,
```

```
    GET_SOURCE_PUBLIC_KEY = 1;
```

```
START REPLICA;
```

```
SHOW REPLICA STATUS\G
```

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> STOP REPLICA;  
Query OK, 0 rows affected (0.13 sec)
```

```
mysql>  
mysql> CHANGE REPLICATION SOURCE TO  
->     SOURCE_HOST = 'mysql-source',  
->     SOURCE_PORT = 3306,  
->     SOURCE_USER = 'repl_user',  
->     SOURCE_PASSWORD = 'Admin12345',  
->     SOURCE_AUTO_POSITION = 1,  
->     GET_SOURCE_PUBLIC_KEY = 1;  
Query OK, 0 rows affected, 2 warnings (0.07 sec)
```

```
mysql>  
mysql> START REPLICA;  
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> SHOW REPLICA STATUS\G
```



mysql> SHOW REPLICA STATUS\G

```
***** 1. row *****
      Replica_IO_State: Waiting for source to send event
        Source_Host: mysql-source
        Source_User: repl_user
        Source_Port: 3306
        Connect_Retry: 60
        Source_Log_File: mysql-bin.000009
      Read_Source_Log_Pos: 198
        Relay_Log_File: relay-bin.000002
        Relay_Log_Pos: 375
      Relay_Source_Log_File: mysql-bin.000009
        Replica_IO_Running: Yes
        Replica_SQL_Running: Yes
        Replicate_Do_DB:
        Replicate_Ignore_DB:
        Replicate_Do_Table:
        Replicate_Ignore_Table:
        Replicate_Wild_Do_Table:
        Replicate_Wild_Ignore_Table:
          Last_Errno: 0
          Last_Error:
          Skip_Counter: 0
        Exec_Source_Log_Pos: 198
          Relay_Log_Space: 580
          Until_Condition: None
          Until_Log_File:
          Until_Log_Pos: 0
        Source_SSL_Allowed: No
        Source_SSL_CA_File:
        Source_SSL_CA_Path:
        Source_SSL_Cert:
        Source_SSL_Cipher:
        Source_SSL_Key:
      Seconds_Behind_Source: 0
Source_SSL_Verify_Server_Cert: No
```


Activity 7 - MySQL Replication using Docker

9. Open new terminal, and execute the command

```
docker exec -it mysql-source mysql -uroot -pAdmin12345
```

```
PS C:\Users\crisp> docker exec -it mysql-source mysql -uroot -pAdmin12345
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.4.8 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```



Activity 7 - MySQL Replication using Docker

9. Inside the **container mysql-source**, the show all running replicas

```
SHOW REPLICAS;
```

```
mysql> SHOW REPLICAS;
+-----+-----+-----+-----+-----+
| Server_Id | Host | Port | Source_Id | Replica_UUID |
+-----+-----+-----+-----+-----+
|          2 |      | 3306 |          1 | f04956e2-3a93-11f1-8d14-0242ac120003 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Activity 7 - MySQL Replication using Docker

10. Test your replication. **Execute the test.sql inside the mysql-source**

```
-- File: ./test.sql
CREATE DATABASE salesdb;
USE salesdb;
CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100),
    price DECIMAL(10,2)
);
INSERT INTO products (product_name, price)
VALUES
('Keyboard', 1500.00),
('Mouse', 800.00),
('Monitor', 9500.00);
```

```
mysql> CREATE DATABASE salesdb;
Query OK, 1 row affected (0.15 sec)

mysql>
mysql> USE salesdb;
Database changed
mysql>
mysql> CREATE TABLE products (
    ->     product_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     product_name VARCHAR(100),
    ->     price DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.09 sec)

mysql>
mysql> INSERT INTO products (product_name, price)
    -> VALUES
    -> ('Keyboard', 1500.00),
    -> ('Mouse', 800.00),
    -> ('Monitor', 9500.00);
Query OK, 3 rows affected (0.13 sec)
Records: 3  Duplicates: 0  Warnings: 0
```



Activity 7 - MySQL Replication using Docker

11. Query the salesdb inside the **mysql-replica**

```
USE salesdb;  
SELECT * FROM products;
```

```
mysql> USE salesdb;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A
```

Database changed

```
mysql> SELECT * FROM products;
```

product_id	product_name	price
1	Keyboard	1500.00
2	Mouse	800.00
3	Monitor	9500.00

```
3 rows in set (0.00 sec)
```

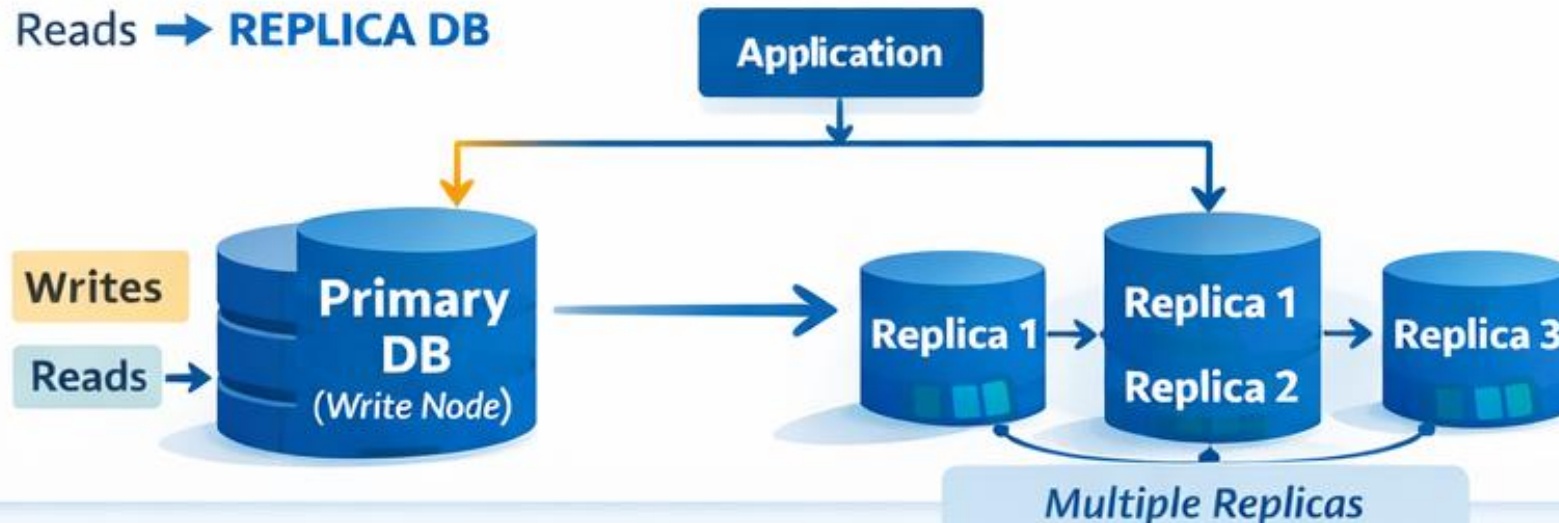
Read scaling

- **Read scaling** in MySQL is about handling **high volumes of read (SELECT) queries efficiently** without overloading a single server.
- This can be implemented by **splitting the workload**
 - **Writes (INSERT, UPDATE, DELETE) → go to Primary (Source)**
 - **Reads (SELECT) → go to Replicas**

Understanding MySQL Read Scaling

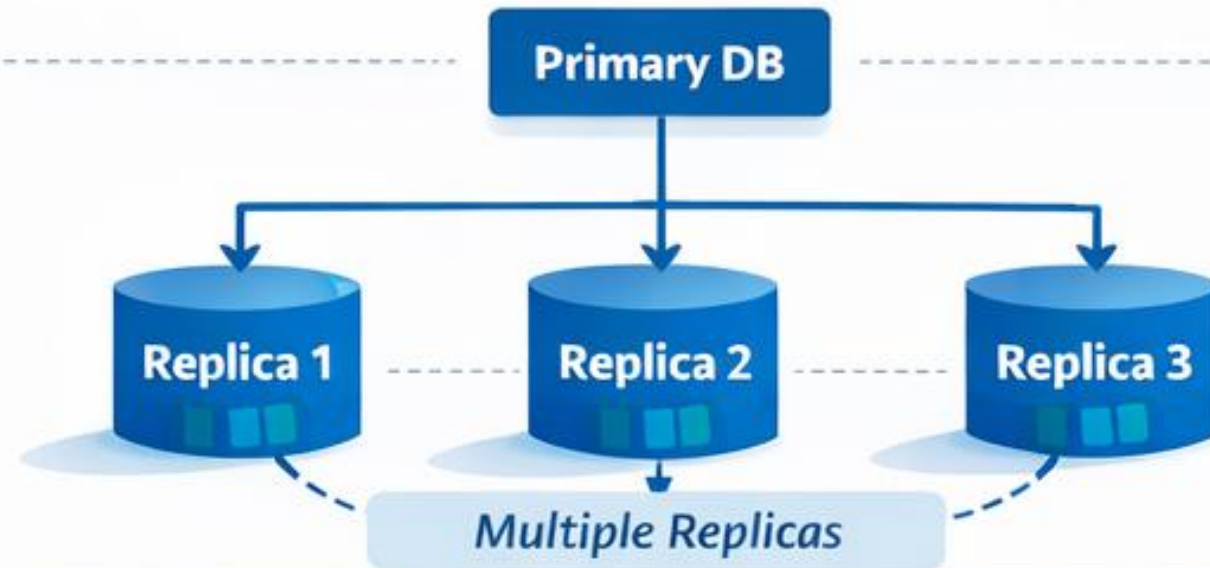
1. Core Idea

- Writes → **PRIMARY DB**
- Reads → **REPLICA DB**



2. Basic Architecture

Split Workload for Efficiency



3. Key Benefits



*Faster
Queries*



*Reduced
Load*



*Better
Scalability*

5. When to Use It?



*Analytics
Reports*



*High Read
Traffic*



Dashboards

4. Challenge

Replication Lag ⚠️

Data Delay Issues!



Failover basics (manual simulation)

- Failover is the process of switching database service from a failed primary server to another server, usually a replica, so the system can continue running.
- **Primary** = accepts writes and usually reads
- **Replica** = copies data from primary
- **Failover** = when primary fails, promote a replica to become the new primary

Failover basics (manual simulation)

1. **Failover is not magic** - Someone or something must decide the primary failed and promote another server.
2. **Replica freshness matters** - If the replica is behind, some recent transactions may be missing.
3. **Promotion is really a role change** - The replica stops replicating and becomes the write server.
4. **Application redirection is required** - Database failover is incomplete if the app still points to the failed server.

Activity 8 - Failover

1. Modify your **docker-compose.yml** and add the snippet below

```
mysql-replica2:
  image: mysql:8.4
  container_name: mysql-replica2
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: Admin12345
  ports:
    - "3309:3306"
  volumes:
    - replica_data2:/var/lib/mysql
    - ./replica2/my.cnf:/etc/my.cnf:ro
  depends_on:
    - mysql-source
  networks:
    - mysql-repl-net
```



Activity 8 - Failover

2. Duplicate the folder and its file **./mysql-replica** into **./mysql-replica2**
3. Update this part of the **./mysql-replica2/my.cnf**

```
[mysqld]  
server-id=3
```

Activity 8 - Failover

4. Start the docker-compose

```
PS D:\AdvancedMySQLProgrammingActivities\Activity7-MySQLReplication> docker compose up -d
[+] Running 4/4
 ✓ Network activity7-mysqlreplication_mysql-repl-net Created
 ✓ Container mysql-source Started
 ✓ Container mysql-replica2 Started
 ✓ Container mysql-replica Started
```

Activity 8 - Failover

5. Add both mysql-replica and mysql-replica2 to the mysql-source

```
mysql> CHANGE REPLICATION SOURCE TO
->     SOURCE_HOST = 'mysql-source',
->     SOURCE_PORT = 3306,
->     SOURCE_USER = 'repl_user',
->     SOURCE_PASSWORD = 'Admin12345',
->     SOURCE_AUTO_POSITION = 1,
->     GET_SOURCE_PUBLIC_KEY = 1;
Query OK, 0 rows affected, 2 warnings (0.06 sec)

mysql>
mysql> START REPLICA;
Query OK, 0 rows affected (0.07 sec)

mysql> SHOW REPLICA STATUS\G
```

Activity 8 - Failover

6. Check replicas from the source (You should see 2 replicas)

```
mysql> SHOW REPLICAS;
```

Server_Id	Host	Port	Source_Id	Replica_UUID
3		3306	1	f72efb84-3ae2-11f1-9d01-0242ac120004
2		3306	1	f7274591-3ae2-11f1-be08-0242ac120003

2 rows in set (0.00 sec)

Activity 8 - Failover

7. Stop the mysql-source container to simulate the process of server down

```
PS D:\AdvancedMySQLProgrammingActivities\Activity7-MySQLReplication> docker stop mysql-source;  
mysql-source
```

Activity 8 - Failover

8. Promote the mysql-replica to become new source

```
STOP REPLICA;  
RESET REPLICA ALL;  
  
SET GLOBAL super_read_only = OFF;  
SET GLOBAL read_only = OFF;  
  
SHOW VARIABLES LIKE 'read_only';  
SHOW VARIABLES LIKE 'super_read_only';
```

```

mysql> STOP REPLICA;
Query OK, 0 rows affected (0.14 sec)

mysql> RESET REPLICA ALL;
Query OK, 0 rows affected (0.07 sec)

mysql> SET GLOBAL super_read_only = OFF;
Query OK, 0 rows affected (0.00 sec)

mysql> SET GLOBAL read_only = OFF;
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW VARIABLES LIKE 'read_only';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| read_only     | OFF   |
+-----+-----+
1 row in set (0.18 sec)

mysql> SHOW VARIABLES LIKE 'super_read_only';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| super_read_only | OFF   |
+-----+-----+
1 row in set (0.00 sec)

```

Activity 8 - Failover

9. Create new replica user inside the new source for the replica client to use.

```
CREATE USER 'repl_user'@'%' IDENTIFIED BY 'Admin12345';  
GRANT REPLICATION SLAVE, REPLICATION CLIENT ON *.* TO 'repl_user'@'%';  
FLUSH PRIVILEGES;
```

Activity 8 - Failover

10. Update the source of the mysql-replica2 into new source

```
STOP REPLICA;  
  
CHANGE REPLICATION SOURCE TO  
    SOURCE_HOST = 'mysql-replica',  
    SOURCE_PORT = 3306,  
    SOURCE_USER = 'repl_user',  
    SOURCE_PASSWORD = 'Admin12345',  
    SOURCE_AUTO_POSITION = 1,  
    GET_SOURCE_PUBLIC_KEY = 1;  
  
START REPLICA;  
SHOW REPLICA STATUS\G
```

```
mysql>
mysql> CHANGE REPLICATION SOURCE TO
->     SOURCE_HOST = 'mysql-replica',
->     SOURCE_PORT = 3306,
->     SOURCE_USER = 'repl_user',
->     SOURCE_PASSWORD = 'Admin12345',
->     SOURCE_AUTO_POSITION = 1,
->     GET_SOURCE_PUBLIC_KEY = 1;
Query OK, 0 rows affected, 2 warnings (0.06 sec)

mysql>
mysql> START REPLICA;
Query OK, 0 rows affected (0.03 sec)

mysql> SHOW REPLICA STATUS\G
***** 1. row *****
      Replica_IO_State: Checking source version
      Source_Host: mysql-replica
      Source_User: repl_user
      Source_Port: 3306
      Connect_Retry: 60
```

Activity 8 - Failover

11. Verify the replicas from the source

```
mysql> SHOW REPLICAS;
```

Server_Id	Host	Port	Source_Id	Replica_UUID
3		3306	2	f72efb84-3ae2-11f1-9d01-0242ac120004

1 row in set (0.00 sec)

mysqldump

- mysqldump is a command-line utility in MySQL used to create logical backups of databases.
- It exports schema (tables, views, procedures) and/or data into a .sql file that you can restore later.

mysqldump

1. Backup a Single Database

➤ `mysqldump -u root -p sales_db > sales_backup.sql`

2. Backup All Databases

➤ `mysqldump -u root -p --all-databases > full_backup.sql`

3. Backup Specific Tables

➤ `mysqldump -u root -p sales_db orders order_items > orders_backup.sql`

mysqldump

4. Backup Only Structure (No Data)

```
mysqldump -u root -p --no-data sales_db > schema.sql
```

5. Backup Only Data (No CREATE TABLE)

```
➤ mysqldump -u root -p --no-create-info sales_db > data.sql
```

Activity 9 - mysqldump

- To create a backup for all databases inside the container

```
docker exec mysql-replica \  
mysqldump -u root \  
--all-databases \  
--single-transaction \  
--routines \  
--triggers \  
--events \  
--set-gtid-purged=ON \  
> full_backup.sql
```



Point-in-time recovery

- Restore your database to an exact moment before a problem happened.
- For example:
 - You took a full backup at **8:00 AM**
 - Someone accidentally deleted rows at **2:15 PM**
 - You want the database back to **2:14 PM**
 - That is **point-in-time recovery**.

Requirements for PITR in MySQL

1. Full backup (**mysqldump**)
2. Binary logging enabled
`[mysqld]`
`log_bin = mysql-bin`
`binlog_format = ROW`
3. To verify if the binlog is enable
`SHOW VARIABLES LIKE 'log_bin';`
`SHOW BINARY LOG STATUS;`

Recovery concept

Step 1: Restore the full backup

```
mysql -u root -p < full_backup.sql
```

Step 2: Find the recovery point

- By date/time
- By binlog position

Recovery concept

Step 3: Replay binlogs up to that point by using **mysqlbinlog**:

- Recover up to a specific time

```
mysqlbinlog --stop-datetime="2026-04-18 14:14:59" mysql-bin.000001 | mysql -u root -p
```

- Recover from one time to another

```
mysqlbinlog \  
--start-datetime="2026-04-18 08:00:00" \  
--stop-datetime="2026-04-18 14:14:59" \  
mysql-bin.000001 | mysql -u root -p
```

- Recover by log position

```
mysqlbinlog --start-position=154 --stop-position=980 mysql-bin.000001 | mysql -u root -p
```

Replication Common Problem

Case 1: IO Thread stopped

➤ **Replica_IO_Running: No**

👉 Cause:

➤ **Wrong credentials**

➤ **Source unreachable**

Replication Common Problem

Case 2: SQL Thread stopped

➤ **Replica_SQL_Running: No**

👉 Cause:

➤ **Duplicate key**

➤ **Data mismatch**

Replication Common Problem

Case 3: Lag increasing

➤ **Seconds_Behind_Source: 300**

👉 Cause:

- Heavy writes on primary
- Slow replica disk

Replication Common Problem

Case 4: Database does not exist

Replica_SQL_Running: No

Last_SQL_Error: Error 'Unknown database 'your_database'' on query. Default database: 'your_database'. Query: 'INSERT INTO ...'

👉 Cause:

➤ Database does not exist in the replica server

SQL Performance and Query Optimization

Query execution flow

- There are two things how the execution flow happen
 1. **Logical query processing order** - this is how SQL is conceptually understood
 2. **Physical execution plan** - this is how MySQL actually executes it internally

Query execution flow

1. Logical query processing order

Even if you write:

```
SELECT ...  
FROM ...  
WHERE ...  
GROUP BY ...  
HAVING ...  
ORDER BY ...  
LIMIT ...
```

SQL does **not** logically
process it top-to-bottom.

MySQL may allow SELECT aliases in HAVING and ORDER BY,
but that does not change the logical query processing order.

The usual logical flow is:

1. FROM
2. JOIN
3. ON
4. WHERE
5. GROUP BY
6. HAVING
7. SELECT
8. DISTINCT
9. ORDER BY
10. LIMIT

Query execution flow

Proof 1: SELECT alias NOT usable in WHERE

ERROR 1054: Unknown column 'total_sales' in 'where clause'

```
SELECT
    product_id,
    SUM(quantity * unit_price) AS total_sales
FROM order_items
WHERE total_sales > 1000    -- ✗ using alias
GROUP BY product_id;
```

Query execution flow

Proof 2: Alias works in HAVING

```
SELECT
    product_id,
    SUM(quantity * unit_price) AS total_sales
FROM order_items
GROUP BY product_id
HAVING total_sales > 1000;    --  works
```


Query execution flow

Proof 3: Aggregates NOT allowed in WHERE

ERROR: Invalid use of group function

```
SELECT
    product_id,
    SUM(quantity * unit_price) AS total_sales
FROM order_items
WHERE SUM(quantity * unit_price) > 1000  -- X
GROUP BY product_id;
```

Query execution flow

Proof 4: ORDER BY CAN use alias

```
SELECT
    product_id,
    SUM(quantity * unit_price) AS total_sales
FROM order_items
GROUP BY product_id
ORDER BY total_sales DESC;  --  works
```

Activity 10 - Logical Processing Interpretation

```
SELECT
    c.customer_id,
    c.first_name,
    SUM(oi.quantity * oi.unit_price) AS total_spent
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
WHERE o.order_date >= '2026-01-01'
GROUP BY c.customer_id, c.first_name
HAVING SUM(oi.quantity * oi.unit_price) > 5000
ORDER BY total_spent DESC
LIMIT 5;
```

Reference

1. FROM
2. JOIN
3. ON
4. WHERE
5. GROUP BY
6. HAVING
7. SELECT
8. DISTINCT
9. ORDER BY
10. LIMIT



Logical vs Actual Execution

- Although the logical order is fixed, MySQL may physically execute it differently for performance.
- For example, MySQL may:
 - Use an index first
 - Reorder joins
 - Push filters earlier
 - Use temporary tables
 - Sort with filesort
 - Materialize derived tables or CTEs

Execution plan analysis

- It answers questions like
 - Which table is read first?
 - Is MySQL using an index or scanning the whole table?
 - How are joins performed?
 - Is sorting or temporary table needed?
 - Which step is expensive?

Execution Plan Tools

- EXPLAIN - Shows the estimated plan.
- EXPLAIN ANALYZE - Actually runs the query and shows
 - Real execution steps
 - Actual timing
 - Actual rows processed

EXPLAIN

```
EXPLAIN
SELECT
    o.order_id,
    o.order_date,
    c.first_name
FROM orders o
JOIN customers c
    ON c.customer_id = o.customer_id
WHERE o.order_date >= '2025-01-01';
```

EXPLAIN

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	o	NULL	ALL	idx_orders_customer_id, idx_orders_order_date	NULL	NULL	NULL	10185	50.0	Using where
1	SIMPLE	c	NULL	eq_ref	PRIMARY	PRIMARY	4	trainingdb.o.customer_id	1	100.0	NULL

EXPLAIN ANALYZ

```
EXPLAIN ANALYZE
SELECT
    o.order_id,
    o.order_date,
    c.first_name
FROM orders o
JOIN customers c
    ON c.customer_id = o.customer_id
WHERE o.order_date >= '2025-01-01';
```

EXPLAIN ANALYZE

```
| -> Nested loop inner join (cost=2809 rows=5092) (actual  
time=0.543..33.2 rows=9500 loops=1)  
    -> Filter: ((o.order_date >= TIMESTAMP'2025-01-01 00:00:00') and  
    (o.customer_id is not null)) (cost=1026 rows=5092) (actual  
time=0.363..7.13 rows=9500 loops=1)  
        -> Table scan on o (cost=1026 rows=10185) (actual  
time=0.358..4.93 rows=10000 loops=1)  
            -> Single-row index lookup on c using PRIMARY  
            (customer_id=o.customer_id) (cost=0.25 rows=1) (actual  
time=0.00236..0.00241 rows=1 loops=9500)  
|
```

Indexing strategies

1. Prefer composite indexes for multi-column filtering

- If queries often filter by multiple columns together, one composite index is usually better than several separate single-column indexes.
- Example query:

```
SELECT *  
FROM orders  
WHERE customer_id = 10  
      AND order_date >= '2025-01-01';  
  
-- Better Index  
CREATE INDEX idx_orders_customer_date  
ON orders (customer_id, order_date);
```



Indexing strategies

2. Use indexes to support joins

- Foreign-key-style columns and join columns are strong index candidates.

Example query:

```
SELECT o.order_id, c.first_name
FROM orders o
JOIN customers c
    ON c.customer_id = o.customer_id;

-- Better Index
CREATE INDEX idx_orders_customer_id
ON orders (customer_id);
```



Indexing strategies

3. Choose a narrow primary key in InnoDB

- In InnoDB, the primary key is the clustered index, so a large primary key affects table storage and secondary indexes too.
- Secondary index entries reference the primary key.

```
-- Good:  
customer_id INT PRIMARY KEY  
  
-- Less ideal:  
email VARCHAR(255) PRIMARY KEY
```

Indexing strategies

4. Avoid indexing every column - Indexes help reads, but they add cost

- INSERT
- UPDATE
- DELETE

Indexing strategies

5. Do not duplicate indexes

-- Bad example:

```
INDEX(customer_id)  
INDEX(customer_id, order_date)
```

-- If you already have:

```
INDEX(customer_id, order_date)
```

Indexing strategies

6. Use covering indexes for high-frequency queries

- A covering index contains all columns needed by the query, so it can answer from the index alone. This can reduce extra row lookups.

```
-- Example:  
SELECT order_id, customer_id, order_date  
FROM orders  
WHERE customer_id = 10;  
  
-- Possible index:  
CREATE INDEX idx_orders_cover_customer  
ON orders (customer_id, order_date, order_id);
```

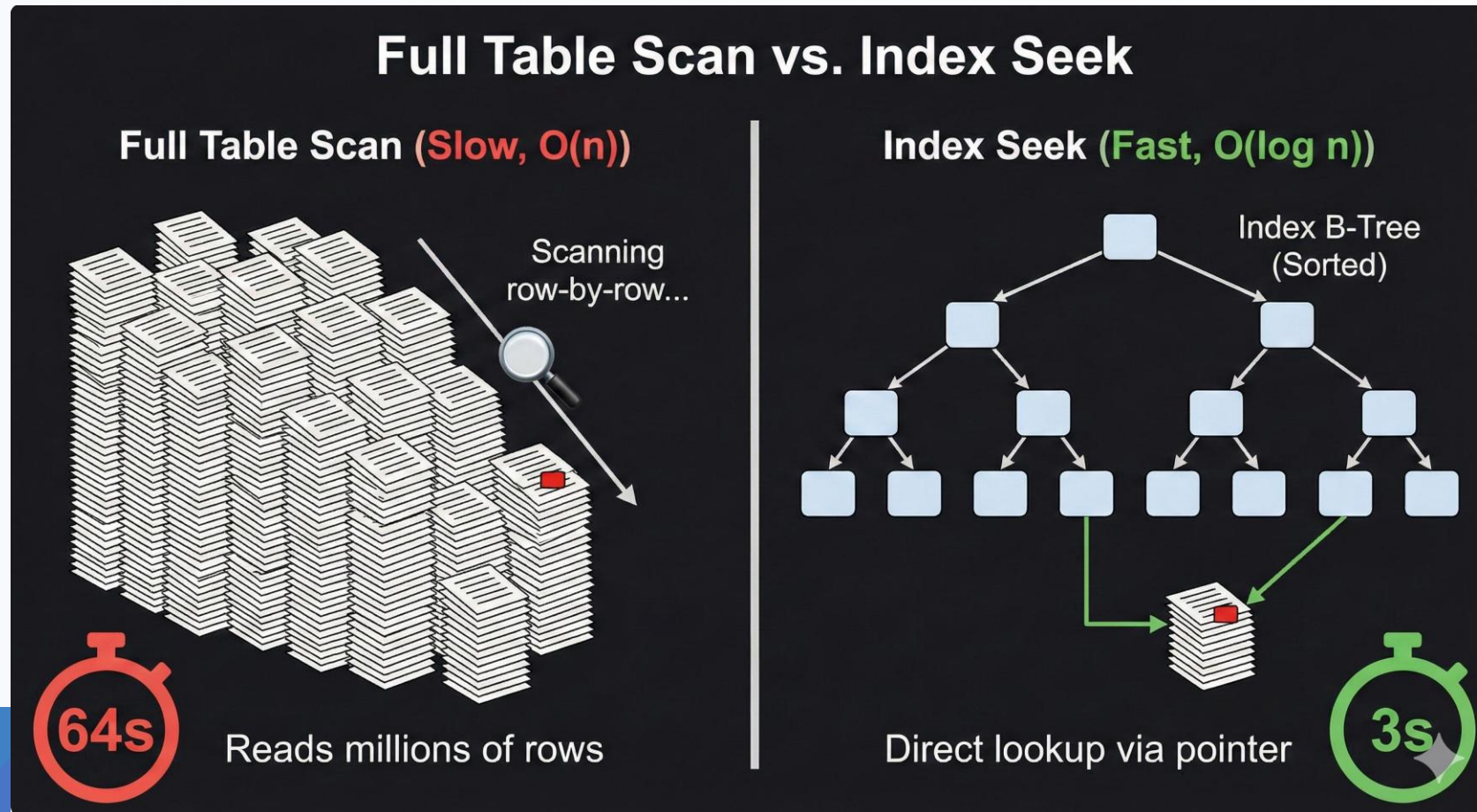

Indexing strategies

9. Match indexes to sorting - If you sort often, an index can help avoid a filesort.

```
-- Example:  
SELECT order_id, order_date  
FROM orders  
WHERE customer_id = 10  
ORDER BY order_date DESC;  
  
-- Possible index:  
CREATE INDEX idx_orders_customer_date_desc  
ON orders (customer_id, order_date DESC);
```

Common Performance Issues

1. Full Table Scan (No Index Usage)



Common Performance Issues

2. Missing Composite Index (Wrong Index Design)

➤ Separate indexes: customer_id and order_date

```
-- Problem
SELECT *
FROM orders
WHERE customer_id = 10
      AND order_date >= '2025-01-01';

-- MySQL may not combine efficiently → still slow
-- Fix
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
```

Common Performance Issues

3. Functions on Indexed Columns - Index on column becomes useless

```
-- Problem
SELECT *
FROM orders
WHERE YEAR(order_date) = 2025;

-- Fix
SELECT *
FROM orders
WHERE order_date >= '2025-01-01'
      AND order_date < '2026-01-01';
```

Common Performance Issues

- 4. SELECT * (Fetching Too Much Data)**
- 5. N+1 Query Problem (Repeated Queries by row-by-row)**
- 6. Correlated Subquery (Executed Per Row)**
- 7. Unnecessary Sorting (ORDER BY without Index)**
- 8. Large OFFSET Pagination**
- 9. Data Type Mismatch**
- 10. Too Many Joins / Bad Join Order**

Optimization techniques

1. Filter early - remove unwanted rows as soon as possible so later operations handle less data.

➤ The fewer rows MySQL processes, the faster the query usually becomes.

2. Reduce columns - Only select the columns you actually need.

3. Rewrite queries - the query is logically correct, but the structure is inefficient. You can often rewrite it to reduce work.

Example of Filter Early

Get top customers (sales > 50,000) but only for recent orders (2025)

```
-- Not optimized (filter late)
SELECT
    o.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
    ON oi.order_id = o.order_id
GROUP BY o.customer_id
HAVING
    SUM(oi.quantity * oi.unit_price) > 50000
    AND MAX(o.order_date) >= '2025-01-01';
```

Example of Filter Early

Get top customers (sales > 50,000) but only for recent orders (2025)

```
-- Optimized (filter early)
SELECT
    o.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
    ON oi.order_id = o.order_id
WHERE o.order_date >= '2025-01-01'
GROUP BY o.customer_id
HAVING SUM(oi.quantity * oi.unit_price) > 50000;
```


Filter Before JOIN

```
-- Filter before join
-- Not optimized
SELECT
    c.customer_id,
    c.first_name,
    o.order_id
FROM customers c
JOIN orders o
    ON o.customer_id = c.customer_id
WHERE c.country = 'USA';
```

Filter Before JOIN

```
-- Better (filter earlier using derived table)
SELECT
    c.customer_id,
    c.first_name,
    o.order_id
FROM (
    SELECT *
    FROM customers
    WHERE country = 'USA'
) c
JOIN orders o
    ON o.customer_id = c.customer_id;
```

Example of Reduce Columns

```
-- Only select the columns you actually need.
-- Not ideal
SELECT *
FROM orders o
JOIN customers c
    ON c.customer_id = o.customer_id;

-- Better
SELECT
    o.order_id,
    o.order_date,
    c.customer_id,
    c.first_name,
    c.last_name
FROM orders o
JOIN customers c
    ON c.customer_id = o.customer_id;
```

Activity 9 - Rewrite Queries

- Suppose you want 2025 orders with customer names and totals.
- Optimized the following query

```
-- Less optimized
SELECT *
FROM customers c
JOIN orders o
    ON o.customer_id = c.customer_id
JOIN order_items oi
    ON oi.order_id = o.order_id
WHERE YEAR(o.order_date) = 2025;
```

MySQL Configuration and Performance Tuning

Understanding MySQL configuration files

- These are MySQL configuration files used to define server settings and behavior.
 - **my.cnf** → Linux / macOS
 - **my.ini** → Windows
- 📁 Same purpose, different file extension

MySQL CNF File Locations

- Linux (my.cnf)
 - /etc/my.cnf
 - /etc/mysql/my.cnf
 - ~/.my.cnf
- Windows (my.ini)
 - C:\ProgramData\MySQL\MySQL Server 8.4\my.ini
- 🖱 MySQL reads multiple config files in order of priority.

MySQL CNF File Structure

- Configuration files are divided into sections (groups):

[mysqld]	-- MySQL Server settings
[client]	-- Client programs (mysql, mysqldump)
[mysql]	-- mysql CLI tool

Example MySQL Configuration

```
[mysqld]
port = 3306
datadir = /var/lib/mysql

# Memory tuning
innodb_buffer_pool_size = 1G

# Connections
max_connections = 200

# Logging
log_error = mysql-error.log
slow_query_log = 1
slow_query_log_file = slow.log

# Replication
server-id = 1
log_bin = mysql-bin
```

Resource management in Docker

- Without limits
 - One container (e.g., MySQL) can consume all RAM/CPU
 - Other services slow down or crash
 - Host machine may become unstable
- With limits
 - Predictable performance
 - Better isolation (like mini-VM behavior)
 - Safer production deployments

CPU Limits in Docker

1. Limit CPU Usage (Quota)

```
docker run -d \  
  --name mysql-db \  
  --cpus="1.5" \  
  mysql:8.4
```

CPU Limits in Docker

2. CPU Shares (Relative Priority)

```
docker run -d \  
  --name mysql-db \  
  --cpu-shares=512 \  
  mysql:8.4
```

CPU Limits in Docker

3. Pin to Specific CPUs

```
docker run -d \  
  --name mysql-db \  
  --cpuset-cpus="0,1" \  
  mysql:8.4
```

Memory Limits in Docker

1. Set Maximum Memory

```
docker run -d \  
  --name mysql-db \  
  --memory="2g" \  
  mysql:8.4
```

Memory Limits in Docker

2. Memory + Swap

```
docker run -d \  
  --name mysql-db \  
  --memory="2g" \  
  --memory-swap="3g" \  
  mysql:8.4
```

Memory Limits in Docker

3. Disable Swap (Strict Control)

`--memory="2g" --memory-swap="2g"`

👉 No swap allowed → safer for databases

Monitoring performance

1. **SHOW STATUS** gives you server activity counters and statistics.

➤ MySQL supports both GLOBAL and SESSION scope, and the statement can be filtered with LIKE or WHERE.

➤ Scope

➤ GLOBAL = server-wide totals

➤ SESSION = only for your current connection

Monitoring performance

-- Show Local and Global Status

SHOW GLOBAL STATUS;

SHOW SESSION STATUS;

-- Connections and threads

SHOW GLOBAL STATUS LIKE 'Connections';

SHOW GLOBAL STATUS LIKE 'Threads_connected';

SHOW GLOBAL STATUS LIKE 'Threads_running';

SHOW GLOBAL STATUS LIKE 'Aborted_connects';

-- Temporary tables and sorts

SHOW GLOBAL STATUS LIKE 'Created_tmp%';

SHOW GLOBAL STATUS LIKE 'Sort%';

-- InnoDB buffer pool

SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool_read%';

SHOW GLOBAL STATUS LIKE 'Innodb_rows_%';

Quick Health Check

```
-- Quick health check
SHOW GLOBAL STATUS
WHERE Variable_name IN (
    'Connections',
    'Threads_connected',
    'Threads_running',
    'Questions',
    'Queries',
    'Created_tmp_tables',
    'Created_tmp_disk_tables',
    'Innodb_buffer_pool_reads',
    'Innodb_buffer_pool_read_requests'
);
```

Quick Health Check

Variable_name	Value
Connections	10
Created_tmp_disk_tables	0
Created_tmp_tables	26
Innodb_buffer_pool_read_requests	345852
Innodb_buffer_pool_reads	1419
Queries	286
Questions	53
Threads_connected	2
Threads_running	2

9 rows in set (0.00 sec)

PERFORMANCE_SCHEMA basics

- The **Performance Schema** is MySQL's built-in low-level monitoring system for measuring server execution instead of guessing.
- It is **enabled by default**, though some collections are only partially enabled depending on setup.

Check if it is enabled

```
SHOW VARIABLES LIKE 'performance_schema';
```

MySQL config:

```
[mysqld]
```

```
performance_schema=ON
```

PERFORMANCE_SCHEMA basics

- The **Performance Schema** is MySQL's built-in low-level monitoring system for measuring server execution instead of guessing.
- It is **enabled by default**, though some collections are only partially enabled depending on setup.

Check if it is enabled

```
SHOW VARIABLES LIKE 'performance_schema';
```

MySQL config:

```
[mysqld]
```

```
performance_schema=ON
```

Basic performance_schema queries

-- A. Top SQL by total execution time

SELECT

digest_text,

count_star,

sum_timer_wait / 1000000000000 **AS** total_seconds,

avg_timer_wait / 1000000000000 **AS** avg_seconds

FROM performance_schema.events_statements_summary_by_digest

ORDER BY sum_timer_wait **DESC**

LIMIT 10;



events_statements_summary_by_digest (10r × 4c)

#	1 digest_text	2 count_star	3 total_seconds	4 avg_seconds
1	SELECT `o`.`customer_id`, SUM(`oi`.`quantity` * `oi`.`unit_price`...	1	0.3611	0.3611
2	SELECT `c`.`customer_id`, `c`.`first_name`, `c`.`last_name`, `o`...	1	0.2287	0.2287
3	SELECT * FROM `customers` `c` JOIN `orders` `o` ON `o`.`customer_`...	1	0.2175	0.2175
4	SELECT `digest_text`, `count_star`, `sum_timer_wait` / ? AS `total_se`...	1	0.2009	0.2009
5	SHOW VARIABLES	2	0.024	0.012
6	SHOW TABLES FROM `information_schema`	2	0.0133	0.0066
7	SHOW GLOBAL STATUS	4	0.0131	0.0033
8	SHOW FUNCTION STATUS WHERE `Db` = ?	2	0.0085	0.0042
9	SELECT `digest_text`, `count_star`, `sum_timer_wait` / ? AS `total_se`...	1	0.0085	0.0085
10	SHOW GLOBAL STATUS LIKE ?	6	0.0082	0.0014



Basic performance_schema queries

-- B. Current active threads

SELECT

thread_id,
processlist_id,
processlist_user,
processlist_host,
processlist_db,
processlist_command,
processlist_time,
processlist_state,
processlist_info

FROM performance_schema.threads

WHERE processlist_command **IS NOT NULL;**

Basic performance_schema queries

1 th...	2 processli...	3 processlist_...	4 processlist_...	5 processlis...	6 processlist_com...	7 processlist...	8 processlist_state	9 processlist_info
56	5	event_scheduler	localhost	(NULL)	Daemon	34,187	Waiting on empty queue	(NULL)
60	7	(NULL)	(NULL)	(NULL)	Daemon	34,187	Suspending	(NULL)
62	9	root	172.17.0.1	trainingdb	Query	0	executing	-- B. Current active threadsSELECT
63	10	root	localhost	(NULL)	Sleep	320	(NULL)	(NULL)

Basic performance_schema queries

```
-- C. File I/O summary
SELECT
    event_name,
    count_star,
    sum_timer_wait / 1000000000000 AS total_seconds
FROM performance_schema.file_summary_by_event_name
ORDER BY sum_timer_wait DESC
LIMIT 10;
```

Basic performance_schema queries

```
the_summary_by_event_name (for x 30)
```

	1 event_name 	2 count_star	3 total_seconds
1	wait/io/file/innodb/innodb_data_file	2,002	1.7144
2	wait/io/file/innodb/innodb_log_file	233	1.421
3	wait/io/file/innodb/innodb_temp_file	70	0.3848
4	wait/io/file/innodb/innodb_dblwr_file	13	0.2018
5	wait/io/file/sql/binlog	33	0.0146
6	wait/io/file/sql/binlog_index	23	0.0063
7	wait/io/file/sql/casetest	15	0.001
8	wait/io/file/sql/misc	5	0.0004
9	wait/io/file/mysys/cnf	5	0.0004
10	wait/io/file/sql/ERRMSG	5	0.0002

Basic performance_schema queries

```
-- D. Status variables via Performance Schema
SELECT *
FROM performance_schema.global_status
WHERE VARIABLE_NAME IN (
    'Threads_connected',
    'Threads_running',
    'Connections'
);
```

Basic performance_schema queries

global_status (3r × 2c)		
#	1 VARIABLE_NAME	2 VARIABLE_VALUE
1	Connections	10
2	Threads_connected	2
3	Threads_running	2



Activity 11 - Using performance_schema

- Find the statements that read many rows to return few rows
- Example output as follows

events_statements_summary_by_digest (10r × 6c)						
#	1 digest_text	2 exec_count	3 sum_rows_examined	4 sum_rows_sent	5 examined_per_row_sent	6 total_seconds
1	SELECT `o`.`customer_id`, SUM(`oi`.`quantity` * ...	1	40,000	264	151.52	0.3611
2	SELECT `c`.`customer_id`, `c`.`first_name`, `c`.`...	1	36,820	6,705	5.49	0.2287
3	SELECT *FROM `customers` `c` JOIN `orders` `o` ON...	1	30,865	20,115	1.53	0.2175
4	SHOW GLOBAL STATUS	4	1,980	1,980	1.0	0.0131
5	SHOW VARIABLES	2	1,292	1,292	1.0	0.024
6	SHOW SESSION STATUS	1	501	501	1.0	0.0023
7	SHOW SESSION STATUS	1	501	501	1.0	0.0025
8	SHOW GLOBAL STATUS	1	495	495	1.0	0.0006
9	SHOW TABLES FROM `information_schema`	2	318	156	2.04	0.0133
10	SELECT `digest_text`, `count_star`, `sum_timer_wait...	2	70	20	3.5	0.2063

Identifying bottlenecks

- Too identify **bottlenecks** in MySQL, the goal is to find **where time is being spent**:
 1. Expensive SQL
 2. Lock waits
 3. I/O waits
 4. Hot tables or indexes
 5. Thread / connection pressure

Identifying bottlenecks

- Find the most expensive queries

```
SELECT
    digest_text,
    count_star AS exec_count,
    ROUND(sum_timer_wait / 1000000000000, 6) AS total_sec,
    ROUND(avg_timer_wait / 1000000000, 6) AS avg_ms,
    sum_rows_examined,
    sum_rows_sent
FROM performance_schema.events_statements_summary_by_digest
ORDER BY sum_timer_wait DESC
LIMIT 10;
```

Identifying bottlenecks

- Find the most expensive queries

events_statements_summary_by_digest (10r x 6c)						
#	1 digest_text	2 exec_count	3 total_sec	4 avg_ms	5 sum_rows_examined	6 sum_rows_sent
1	SELECT `o`.`customer_id`, SUM(`oi`.`quantity` * ...	1	0.3611	361.0613	40,000	264
2	SELECT `c`.`customer_id`, `c`.`first_name`, `c`...	1	0.2287	228.6966	36,820	6,705
3	SELECT * FROM `customers` `c` JOIN `orders` `o` ON...	1	0.2175	217.522	30,865	20,115
4	SELECT `digest_text`, `count_star`, `sum_timer_wai...	2	0.2063	103.131	70	20
5	SELECT `thread_id`, `processlist_id`, `processlist_use...	1	0.1092	109.1561	51	4
6	SHOW VARIABLES	2	0.024	12.0037	1,292	1,292
7	SELECT `event_name`, `count_star`, `sum_timer_wai...	1	0.0214	21.3611	61	10
8	SHOW TABLES FROM `information_schema`	2	0.0133	6.6398	318	156
9	SHOW GLOBAL STATUS	4	0.0131	3.2788	1,980	1,980
10	SHOW FUNCTION STATUS WHERE `Db` = ?	2	0.0085	4.2495	4	0

Identifying bottlenecks

- Find wait bottlenecks by event type

```
SELECT
    event_name,
    count_star,
    ROUND(sum_timer_wait / 1000000000000, 6) AS total_sec,
    ROUND(avg_timer_wait / 1000000000, 6) AS avg_ms
FROM performance_schema.events_waits_summary_global_by_event_name
WHERE sum_timer_wait > 0
ORDER BY sum_timer_wait DESC
LIMIT 15;
```

Identifying bottlenecks

- Find the most expensive queries

events_waits_summary_global_by_event_name (15r x 4c)				
#	1 event_name	2 count_star	3 total_sec	4 avg_ms
1	idle	919	21,418.7212	23,306.552
2	wait/io/file/innodb/innodb_data_file	2,002	1.7144	0.8563
3	wait/io/file/innodb/innodb_log_file	233	1.421	6.0987
4	wait/io/table/sql/handler	108,688	0.5363	0.0049
5	wait/io/file/innodb/innodb_temp_file	70	0.3848	5.4967
6	wait/io/file/innodb/innodb_dblwr_file	13	0.2018	15.5194
7	wait/io/file/sql/binlog	33	0.0146	0.4426
8	wait/io/file/sql/binlog_index	23	0.0063	0.2749
9	wait/io/file/sql/casetest	15	0.001	0.0665
10	wait/io/file/sql/misc	5	0.0004	0.0845
11	wait/io/file/mysys/cnf	5	0.0004	0.0797
12	wait/io/file/sql/ERRMSG	5	0.0002	0.0443



Identifying bottlenecks

- Identify table I/O hotspots

```
-- Find which tables are getting the most reads and writes.
SELECT
    object_schema,
    object_name,
    count_fetch AS `reads`,
    count_insert AS `inserts`,
    count_update AS `updates`,
    count_delete AS `deletes`,
    ROUND(sum_timer_fetch / 1000000000000, 6) AS read_sec,
    ROUND(sum_timer_insert / 1000000000000, 6) AS insert_sec,
    ROUND(sum_timer_update / 1000000000000, 6) AS update_sec,
    ROUND(sum_timer_delete / 1000000000000, 6) AS delete_sec
FROM performance_schema.table_io_waits_summary_by_table
ORDER BY (
    sum_timer_fetch + sum_timer_insert + sum_timer_update + sum_timer_delete
) DESC LIMIT 10;
```



Identifying bottlenecks

- Identify table I/O hotspots

table_io_waits_summary_by_table (10r × 10c)										
#	1 object_schema	2 object_name	3 reads	4 inserts	5 updates	6 deletes	7 read_sec	8 insert_sec	9 update_sec	10 delete_sec
1	trainingdb	order_items	70,230	0	0	0	0.3472	0.0	0.0	0.0
2	trainingdb	customers	7,706	0	0	0	0.1355	0.0	0.0	0.0
3	trainingdb	orders	30,752	0	0	0	0.0537	0.0	0.0	0.0
4	performance_schema	table_io_waits_summary_by_table	0	0	0	0	0.0	0.0	0.0	0.0
5	mysql	schemata	0	0	0	0	0.0	0.0	0.0	0.0
6	mysql	tablespace_files	0	0	0	0	0.0	0.0	0.0	0.0
7	mysql	tablespaces	0	0	0	0	0.0	0.0	0.0	0.0
8	mysql	check_constraints	0	0	0	0	0.0	0.0	0.0	0.0
9	mysql	column_type_elements	0	0	0	0	0.0	0.0	0.0	0.0
10	mysql	foreign_key_column_usage	0	0	0	0	0.0	0.0	0.0	0.0



Understanding MySQL Logs

Types of logs

Log Type	Purpose	When to Use
Error Log	Server startup, shutdown, crashes, errors	Debug failures
General Query Log	All SQL statements received	Debug application behavior
Slow Query Log	Queries that exceed time threshold	Performance tuning
Binary Log (binlog)	Data changes (INSERT/UPDATE/DELETE)	Replication & recovery
Relay Log	Replica-side copy of binlog	Replication internals

Error Log (Most Important for Stability)

- What it contains
 - Startup/shutdown messages
 - Crashes
 - InnoDB errors
 - Replication errors
- When to check
 - MySQL won't start
 - Replication stops
 - Unexpected shutdowns

Error Log (Most Important for Stability)

- Example
`log_error = mysql-error.log`
- If your config also has: `datadir = /var/lib/mysql`
 - Then the error log will be: `/var/lib/mysql/mysql-error.log`
- Sample content
`[ERROR] InnoDB: Unable to lock ./ibdata1`
`[Warning] Aborted connection 123 to db: 'test'`
- Docker: `docker logs mysql-container`

General Query Log (Full Visibility)

- It logs every SQL query executed
- To enable
 - **Command:**
`SET GLOBAL general_log = 'ON';`
 - **.CNF/.INI:**
`general_log = 1`
`general_log_file = general.log`

Sample output

Query SELECT * FROM orders;

Query INSERT INTO customers VALUES (...);

Slow Query Log (Performance Goldmine)

- This is the MOST IMPORTANT for optimization.
- Use case
 - Detect missing indexes
 - Identify full table scans
 - Optimize JOINS

Slow Query Log (Performance Goldmine)

Metric	Meaning
Query_time	Execution time
Rows_examined	Rows scanned (inefficiency indicator)
Rows_sent	Rows returned

Slow Query Log (Performance Goldmine)

- To Enable:

- Command

- ```
SET GLOBAL slow_query_log = 1;
SET GLOBAL long_query_time = 2; -- seconds
```

- .Cnf/.INI:

- ```
slow_query_log = 1  
slow_query_log_file = slow.log  
long_query_time = 2
```

Sample Output

Query_time: 5.234 Rows_examined: 100000
SELECT * FROM orders WHERE customer_id = 100;

Binary Log (binlog) – Replication & Recovery

- It logs Data changes only (DML/DDDL)
- .CNF/.INI Configuration:
 - log_bin = mysql-bin -- To enable
 - binlog_format = ROW -- recommended
 - binlog_format = STATEMENT -- Full statement
 - binlog_format = MIXED -- Hybrid
- Sample Output:
 - ### INSERT INTO orders
 - ### UPDATE customers SET name='John'

Analyzing slow queries using mysqldumpslow

1. Show the log location

```
mysql> SHOW VARIABLES LIKE 'slow_query_log_file';
```

Variable_name	Value
slow_query_log_file	/var/lib/mysql/f08abab5430e-slow.log

Analyzing slow queries using mysqldumpslow

- Sort by average execution time (default):
`mysqldumpslow /var/log/mysql/slow.log`
- Show the top 10 slowest queries, sorted by total execution time (-s t):
`mysqldumpslow -t 10 -s t /var/log/mysql/slow.log`
- Show top 5 queries, sorted by total rows examined (-s r):
`mysqldumpslow -t 5 -s r /var/log/mysql/slow.log`
- Show only the top 10 longest-running queries (no sorting):
`mysqldumpslow -t 10 /var/log/mysql/slow.log`

Analyzing slow queries using mysqldumpslow

2. Analyze your slow query logs using **mysqldumpslow**

- Sort by average execution time (default)

`mysqldumpslow /var/log/mysql/slow.log`

- Show the top 10 slowest queries, sorted by total execution time (-s t):

`mysqldumpslow -t 10 -s t /var/log/mysql/slow.log`

- Show top 5 queries, sorted by total rows examined (-s r):

`mysqldumpslow -t 5 -s r /var/log/mysql/slow.log`

- Show only the top 10 longest-running queries (no sorting):

`mysqldumpslow -t 10 /var/log/mysql/slow.log`

Log rotation and management

- For general query log and slow query log, the usual file-based rotation pattern is:

```
-- Linux/Unix shell idea
mv general.log general-old.log
mysqladmin flush-logs general

mv slow.log slow-old.log
mysqladmin flush-logs slow
```

Log rotation and management

- You can also rotate the general query log by turning it off, changing the log file name, then turning it back on:

```
SET GLOBAL general_log = 'OFF';  
SET GLOBAL general_query_log_file = '/path/to/general-2026-04-19.log';  
SET GLOBAL general_log = 'ON';
```

Log rotation and management

- For the error log, MySQL supports flushing it too:
 - FLUSH ERROR LOGS;
- Or more broadly:
 - FLUSH LOGS;
 - 🖱 Affects ALL log types
 - Error log
 - General log
 - Slow query log
 - Binary logs

Important difference vs other logs

Log Type	Flush Behavior
General / Error / Slow	Close + reopen same file
Binary Logs	Close current + create new file

Avoiding Metadata Locks and Concurrency Issues

What are metadata locks (MDL)

- **Metadata Locks (MDL)** in MySQL are *internal locks* that protect the **structure (metadata)** of database objects like tables, views, and schemas.

What are metadata locks (MDL)

- ☒ **Someone is reading** → others can also read
- ☒ **Someone is editing structure (like renaming columns)** → nobody else can read/write
- MDL ensures this consistency.

When MDL Happens

1. **SELECT (Read)** - Acquires a shared MDL (read lock)

- Allows:

- ☒ Other SELECTs
- ☒ INSERT/UPDATE/DELETE

- Blocks:

- ☒ ALTER TABLE
- ☒ DROP TABLE

When MDL Happens

2. **DML** (INSERT / UPDATE / DELETE)

- ☒ Also uses shared MDL
- ☒ Same behavior as SELECT

When MDL Happens

3. **DDL (ALTER / DROP / RENAME)** - Requires exclusive MDL

✗ Blocks ALL queries on the table and must wait until all active queries finish

Scenario of Locking

```
-- Session 1
START TRANSACTION;
SELECT * FROM orders;  -- holds MDL

-- Session 2
ALTER TABLE orders ADD COLUMN test INT;  -- waiting...
```

```
65 -- Session 1  START TRANSACTION;
66 SELECT * FROM orders;
67 -- holds MDL;
68 /* Affected rows: 0  Found rows: 10,000  Warnings: 0  Duration for 3 queries: 0.000 sec. */
```

```
Database changed
mysql> ALTER TABLE orders ADD COLUMN test INT; -- waiting...
```

Check metadata locks

```
-- Check metadata locks
```

```
SELECT *
```

```
FROM performance_schema.metadata_locks
```

```
WHERE OBJECT_TYPE = 'TABLE';
```

metadata_locks (2r x 11c)									
	1 OBJECT_TYPE	2 OBJECT_SCHEMA	3 OBJECT_NAME	4 COLUMN_NAME	5 OBJECT_INSTANCE_BEGIN	6 LOCK_TYPE	7 LOCK_DURATION	8 LOCK_STATUS	9 SOURCE
1	TABLE	trainingdb	orders	(NULL)	139,682,136,458,240	SHARED_READ	TRANSACTION	GRANTED	sql_parse.cc:6178
2	TABLE	performance_schema	metadata_locks	(NULL)	139,682,136,974,944	SHARED_READ	TRANSACTION	GRANTED	sql_parse.cc:6178



See blocking queries

SELECT

```
t.PROCESSLIST_ID,  
t.PROCESSLIST_INFO,  
ml.OBJECT_SCHEMA,  
ml.OBJECT_NAME,  
ml.LOCK_TYPE,  
ml.LOCK_STATUS
```

```
FROM performance_schema.metadata_locks ml  
JOIN performance_schema.threads t  
ON ml.OWNER_THREAD_ID = t.THREAD_ID;
```

Result #1 (3r x 6c)						
#	1 PROCESSLIST_ID	2 PROCESSLIST_INFO	3 OBJECT_SCHEMA	4 OBJECT_NAME	5 LOCK_TYPE	6 LOCK_STATUS
1	12	-- See blocking queriesSELECT t.PROCESSLIS...	trainingdb	orders	SHARED_READ	GRANTED
2	12	-- See blocking queriesSELECT t.PROCESSLIS...	performance_schema	metadata_locks	SHARED_READ	GRANTED
3	12	-- See blocking queriesSELECT t.PROCESSLIS...	performance_schema	threads	SHARED_READ	GRANTED

Common causes

- Starts a transaction and does not COMMIT or ROLLBACK
- Runs a long statement
- Holds an MDL or row lock

Row Lock Vs Table Lock

Feature	Row Lock	Table Lock
Scope	Specific rows	Entire table
Concurrency	High	Low
Performance	Better for OLTP	Can block heavily
Engine	InnoDB	MyISAM (or fallback)

Table Locking

A. No index - many rows locked (looks like table lock)

B. LOCK TABLES (explicit table lock)

```
LOCK TABLES orders WRITE, customers READ;
```

Table	Your session	Other sessions
orders WRITE	Read ☒ Write ☒	Read ☒ Write ☒
customers READ	Read ☒ Write ☒	Read ☒ Write ☒

Row Lock - Race Condition Prevention

```
-- This is used for safe read → then update logic
```

```
START TRANSACTION;
```

```
SELECT status
```

```
FROM orders
```

```
WHERE order_id = 100
```

```
FOR UPDATE;
```

```
-- check status in app logic
```

```
-- Imagine your app does:
```

```
-- IF status = 'Pending' THEN allow shipping
```

```
UPDATE orders
```

```
SET status = 'SHIPPED'
```

```
WHERE order_id = 100;
```

```
COMMIT;
```



Releasing Lock

- Let say, there are transaction and does not COMMIT or ROLLBACK, runs a long statement, holds and MDL or row lock.
- For example,

```
START TRANSACTION;  
  
SELECT status  
FROM orders  
WHERE order_id = 100  
FOR UPDATE;
```

Releasing Lock

- To force release
 1. Show the process list (Tab 1)
SHOW PROCESSLIST;

```
mysql> SHOW PROCESSLIST;
```

Id	User	Host	db	Command	Time	State	Info
5	event_scheduler	localhost	NULL	Daemon	46532	Waiting on empty queue	NULL
10	root	localhost	trainingdb	Query	0	init	SHOW PROCESSLIST
12	root	172.17.0.1:54406	trainingdb	Sleep	8		NULL
13	root	localhost	trainingdb	Sleep	16		NULL

```
4 rows in set, 1 warning (0.00 sec)
```



```
mysql> SELECT CONNECTION_ID();
```

CONNECTION_ID()
10

```
1 row in set (0.00 sec)
```



Releasing Lock

- To force release

1. Show the process list (Tab 2)

```
mysql> SELECT * FROM orders
-> WHERE order_id = 100
-> FOR UPDATE;
+-----+-----+-----+-----+
| order_id | customer_id | order_date       | status |
+-----+-----+-----+-----+
|      100 |          101 | 2026-01-05 10:34:44 | SHIPPED |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT CONNECTION_ID();
+-----+
| CONNECTION_ID() |
+-----+
|          13 |
+-----+
1 row in set (0.00 sec)
```

Releasing Lock

2. Kill only the running query

KILL QUERY 13;

```
mysql> KILL CONNECTION 13;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SHOW PROCESSLIST;
```

Id	User	Host	db	Command	Time	State	Info
5	event_scheduler	localhost	NULL	Daemon	46797	Waiting on empty queue	NULL
10	root	localhost	trainingdb	Query	0	init	SHOW PROCESSLIST
12	root	172.17.0.1:54406	trainingdb	Sleep	13		NULL

```
3 rows in set, 1 warning (0.00 sec)
```

Types of Row Locks in InnoDB

1. Shared Lock (S Lock)

- Used for reading
- Multiple transactions can read the same row
- No one can modify it

```
-- Shared Lock (S Lock)  
START TRANSACTION;  
SELECT * FROM orders  
WHERE order_id = 100  
LOCK IN SHARE MODE;  
COMMIT;
```


Types of Row Locks in InnoDB

2. Exclusive Lock (X Lock)

- Used for writing (INSERT, UPDATE, DELETE)
- Only one transaction can hold it

```
-- Exclusive Lock (X Lock)
START TRANSACTION;
SELECT * FROM orders
WHERE order_id = 100
FOR UPDATE;
COMMIT;
```

Danger with Row Locks

- If the order_date column is NOT indexed, MySQL will scan the table and locks many rows

```
START TRANSACTION;  
UPDATE orders  
SET status = 'SHIPPED'  
WHERE order_date = '2025-01-01';
```

Strategies to avoid blocking

1. Keep transactions short

- Do not leave a transaction open while waiting for user input, app logic, API calls, or screen interaction.

2. Use proper indexes

- If your WHERE clause is not indexed, MySQL may scan many rows and lock more than you expected.

3. Lock only when needed

- A plain SELECT is usually a consistent nonlocking read in InnoDB.

4. Access tables and rows in a consistent order

- Example: always update customers first; or always lock smaller order_id first.



Strategies to avoid blocking

5. Use NOWAIT when you prefer fast failure over waiting

- SELECT ... FOR UPDATE NOWAIT returns immediately with an error if the row is already locked.
- This is useful for app flows where “try again later” is better than hanging.

```
SELECT *  
FROM orders  
WHERE order_id = 100  
FOR UPDATE NOWAIT;
```

Strategies to avoid blocking

6. Use SKIP LOCKED for queue-style processing

- Good for job queues, task pickup, batch workers.
- Not ideal for normal business queries because MySQL warns it returns an inconsistent view of the data.

```
SELECT order_id
FROM orders
WHERE status = 'PENDING'
ORDER BY order_id
LIMIT 10
FOR UPDATE SKIP LOCKED;
```

Strategies to avoid blocking

7. Avoid unnecessary LOCK TABLES
8. Be careful with DDL during busy hours
9. Statements like ALTER TABLE use metadata locking (MDL).
10. Choose the right isolation level
11. Retry on deadlocks and handle lock timeouts
12. Deadlocks are normal in concurrent systems.

Pattern: Shorter Transactions

```
-- Good Pattern
START TRANSACTION;

SELECT status
FROM orders
WHERE order_id = 100
FOR UPDATE;

UPDATE orders
SET status = 'SHIPPED'
WHERE order_id = 100;

COMMIT;
```

```
-- Bad pattern:
START TRANSACTION;

SELECT status
FROM orders
WHERE order_id = 100
FOR UPDATE;

-- wait for user click
-- call external API
-- sleep
-- more app logic

UPDATE orders
SET status = 'SHIPPED'
WHERE order_id = 100;

COMMIT;
```



Online DDL

(**ALGORITHM=INPLACE, LOCK=NONE**)

- **ALGORITHM=INSTANT** leveraging row versioning and the Transactional Data Dictionary to avoid touching any physical data.
- **ALGORITHM=COPY** is the most expensive and uses lock table
- **ALGORITHM=INPLACE** it perform the schema change in-place
 - For example, certain MODIFY COLUMN, ROW_FORMAT, or character set changes can still reorganize data substantially.
- **LOCK=NONE** it allow concurrent DML as much as possible, so other sessions can usually still SELECT, INSERT, UPDATE, and DELETE while the DDL runs

Online DDL Algorithm Comparison

Algorithm	Speed	Rebuilds Table?	Concurrent Writes (DML)?	Use Case
INSTANT	Milliseconds	No	Yes	Adding columns, changing defaults
INPLACE	Medium	Sometimes	Yes	Adding indexes, some column modifications
COPY	Slow	Always	No	Heavy structural changes (e.g., changing data types)

Online DDL Limitations

1. It is a request, not a guarantee.
2. LOCK=NONE does not mean no metadata locking.
3. Some operations are still expensive.
4. Some tables/features restrict LOCK=NONE.

Scenario for ALGORITHM=INSTANT

-- Goal: Add a new column to the orders table with minimal disruption.

```
ALTER TABLE orders  
ADD COLUMN remarks VARCHAR(255) NULL,  
ALGORITHM=INSTANT;
```

Scenario for ALGORITHM=COPY

```
-- Goal: Add an index on order_date so reports run faster, while keeping  
the table available.
```

```
ALTER TABLE orders  
ADD INDEX idx_orders_order_date (order_date),  
ALGORITHM=INPLACE,  
LOCK=NONE;
```

Scenario for ALGORITHM=COPY

-- Goal: Change a column definition in a way that requires MySQL to rebuild the table.

```
ALTER TABLE products  
MODIFY COLUMN product_name VARCHAR(500) NOT NULL,  
ALGORITHM=COPY;
```

Activity 11 - Online DDL

- Scenario 1: The business wants to store a **customer** loyalty tier (BRONZE, SILVER, GOLD), but this is optional for now.
- Task:
 - Modify the customers table to add a column loyalty_tier
 - Must NOT block reads/writes
 - Must fail if it cannot be done instantly
 - Should default to NULL

Activity 11 - Online DDL

- Scenario 2: A reporting query is slow:

```
SELECT *  
FROM orders  
WHERE customer_id = 100  
AND order_date >= '2025-01-01';  
Question
```

- Optimize this by adding an index:
 - Must allow concurrent reads and writes
 - Must NOT rebuild the table fully

Activity 11 - Online DDL

- Scenario 3: The **products.product_name** column is too small and needs to be increased from **VARCHAR(100)** to **VARCHAR(500)**.
- Task:
 - Write the ALTER statement that forces MySQL to reveal if this is unsafe.

Safe schema changes in production

- **Prefer online DDL**

- Use operations that can run with `ALGORITHM=INSTANT` or `ALGORITHM=INPLACE` instead of `COPY`, because `COPY` is the most disruptive

- **Prefer low-lock changes**

- When supported, use `LOCK=NONE` so reads and writes can continue during the change.

- **Test the exact ALTER first**

- MySQL documents that omitted `ALGORITHM` may use `INSTANT`, then `INPLACE`, then `COPY` if needed.

A practical production workflow...

1. Take a backup or ensure restore capability exists.
2. Run the exact ALTER TABLE in staging with production-like data size.
3. Check whether the statement can use INSTANT or INPLACE.
4. Schedule during low traffic if the table is hot.
5. Watch for metadata lock waits, long-running transactions, and replica lag.
6. Apply to replicas carefully if needed, especially for large rebuild operations.

Copyright Protected

- Copyright © 2024 Cris Perando [crisperando@gmail.com]. All Rights Reserved.
- No part of this work may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the copyright holder, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.
- TrainOSys is a registered trademark of www.trainosys.com.
- All other trademarks mentioned herein are the property of their respective owners.



TRAINOSYS
Training the Future Today

Reach Us!

Visit Us

12th/F The Trade & Financial Tower Unit 1206
32nd Street & 7th Avenue Bonifacio Global City,
Taguig 1634 Philippines

Email Us

inquiry@trainosys.com

Browse Our Website

www.trainosys.com

